INDIVIDUAL CAPSTONE REPORT

Title: Building a simulation for Fixed-Wing UAV using Reinforcement Adaptive Co-Piloting

Student Name: Rija Bhomi

University ID: 0371405

Date:

1st April, 2026

Table of Contents

CHATPER 2: RELATED WORKS 1
2.1 Background Review 1
2.1.1 Context & Significance 1
2.1.2 Existing Systems: 1
2.2 Detailed Paper Summaries 3
2.2.1 GCS Dashboard & Visualization Architecture 3
2.2.2 Telemetry Data Pipeline & Logging 6
2.2.3 Model Optimization and Reward Function 7
2.3 Methodology, Technology, and Tools Analysis 9
2.3.1 Data Pipeline Architecture: Storage and Structure 9
2.3.2 Telemetry Protocol: MAVLink & Transport Layer 10
2.3.3 Reward Function Design: Continuous vs Sparse 10
2.4 Summary of Key Findings and Research Gaps 11
2.4.1 Key Findings 11
2.4.2 Identified Research Gaps 12
2.5 Summary Table of Past Works 13
Conclusion 14
CHAPTER 3: SYSETEM ANALYSIS 15
3.1 SDLC Methodology: SCRUM Framework 15
3.1.1 SCRUM Roles 15
3.1.2 Sprint Structure 16
3.2 Theoretical: SWOT Analysis 17
3.2.1 Programming Languages: 17
3.2.2 Data Storage 18
3.2.3 Communication Protocols 19
3.2.4 Frameworks 19
3.2.5 ML Framework 20
3.3 Technical Analysis 20
3.3.1 Libraries and Tools 20
3.3.2 GCS Interface & Telemetry Mapping 21
3.3.3 Reward Function Implementation 22

3.4 Requirement Specifications 22
3.4.1 Functional Requirements 22
3.4.2 Non-functional Requirements 23
CHAPTER 4: System Design 23
4.1 System Architecture Diagram 23
4.2 Use Case Diagrams 27
4.3 Activity Diagrams 31
4.4 Sequence Diagrams 37
References 41
Appendix 42

Figure 1: Data Flow Architecture 24
Figure 2: System Architecture 25
Figure 3: Use Case: Human Pilot 27
Figure 4: Use Case Diagram: AI co-pilot 28
Figure 5: Use Case Diagram: System Admin 29
Figure 6: Use Case Diagram: System 30
Figure 7: Activity Diagram- Manual Flight Control 31
Figure 8: Activity Diagram: AI co-pilot Stability Agumentation 32
Figure 9: Activity Diagram: Control Handover 33
Figure 10: Activity Diagram- Reward function design and Model Optimization 34
Figure 11: Activity Diagram- Telemetry Data Pipeline and Logging 35
Figure 12: Activity Diagram- Dashboard Visualization 36
Figure 13: Sequence Diagram- Telemetry Data Pipeline and logging 37
Figure 14: Sequence Diagram- Reward Function calculation and Model Optimization 38
Figure 15: Sequence Diagram: Control Handover between Human pilot and AI co-pilot 39
Figure 16: Sequence Diagram- System Initialization and startup sequence 40

Table 1: Data Storage Selection 9
Table 2: Telemetry Protocol Selection 10
Table 3: Reward Function Design Choices 11
Table 4: Observed Limitations, Research Gap, and Impact on Proposed Project 12
Table 5: Summary of Past works 13
Table 6: SCRUM Roles 15
Table 7: Sprint Structure 16
Table 8: Detailed Sprint Breakdown 16
Table 9: Milestone Tracking 17

Table 10: SWOT analysis of Programming Languages 17
Table 11: SWOT analysis of Data Storage 18
Table 12: SWOT analysis of Communication Protocols 19
Table 13: SWOT analysis of Frameworks used 19
Table 14: SWOT analysis of ML Framework 20
Table 15: Libraries and Tools used 21
Table 16: GCS interface and Telemetry mapping 21
Table 17: Reward Function Implementation 22
Table 18: Functional Requirements 22
Table 19: Non-functional Requirements 23

CHAPTER 2: RELATED WORKS

2.1 Background Review

This section represents a summary of existing systems that are relevant to the proposed work, providing critical analysis of their strengths, limitations and how they guide design decisions for this capstone project. The review focuses on three main domains: Ground Control Station (GCS) interfaces, telemetry data pipeline systems and reinforcement learning frameworks for UAV control.

2.1.1 Context & Significance

In the recent years, there has been rapid advancement in Unmanned Aerial Systems (UAVs) in civilian and commercial sectors. This advancement brings the need of shifting from basic flight capability like: manual control to hybrid autonomous systems. However, for these systems to work well, several things like: flight mechanics of drone, data pipelines, visualization in the Ground Control Station (GCS) and reward structure used in Reinforcement Learning must function properly. As UAV operations are becoming more complex, the role of data architect and model optimizer becomes very crucial to ensure correct logging of telemetry, clear visualization and to train stable AI co-pilots.

This literature review focuses on three main domains relevant to capstone project: architecture and visualization of GCS, telemetry data pipelines and logging, and reward function optimization. There is a gap between high-fidelity simulation data and real-useful actionable insights that human operators and AI agents. So, the key research problem is to address this disconnection between simulation data and actionable insights and addressing the trust gap between human operator and autonomous agents. Existing research often treat UI design, data logging and control algorithms separately which causes data latency and creation of poor reward function that works in simple simulation but fails in real environment.

2.2 Existing Systems:

a. Ground Control Station (GCS) Systems

(Hubsky, 2024) evaluated Ground Control Station (GCS) interface design by developing a comprehensive analysis system and evaluated existing GCS platforms like Mission Planner and QGroundControl. The research mainly focused on evaluation of usability, operator efficiency issues, difficulty understanding flight information and poor situational awareness.

Critical Analysis: The study identifies many usability problems in the existing GCS interface like Mission Planer are very powerful but also very cluttered showing too much information that makes it harder for operator to quickly understand important data. While the system provides thorough documentation of UI short comes, it does not analyze the underlying data architecture and backend that can reduce frontend complexity. The research does not consider integration of AI-generated telemetry data into the system.

Relevance to the project: This supports the concept that GCS system must balance data presentation with system efficiency which our project is trying to address by focusing on data

architecture perspective by improving structured telemetry logging, and analytical dashboard separation. Unlike (Hubsky, 2024) focus on operator-facing dashboard, our project also focuses on data architecture and analytical monitoring. The system will have structure telemetry data efficiently in the backend, separate analytical dashboards from operational displays and allow real-time visualization which connects data integrity with data visualization.

b. Telemetry Data Pipeline and Logging Architectures

(Aziz & Loya, 2025) developed a system called AeroQT which is a lightweight GCS application that connects JSBSim and MAVLink which allows engineers to run simulated flight tests. This system supports two types of testing: Software-in-the-loop and Hardware-in-the-Loop that allows engineers to evaluate aircraft performance without modifying or risking real aircraft.

Critical Analysis: While the system achieves successful integration of JSBSim with MAVLink communication protocols, the system experiences a telemetry delay of approximately 400ms which reduces the bandwidth of real-time controllers. The system was designed mainly for single-aircraft testing and does not support multi-UAV operations, and advanced mission planning. Moreover, the paper mentions that logging system still need improvement which suggests that the system did not focus heavily on data structuring, timestamp synchronization and telemetry pipeline design.

Relevance to the project: This supports the concept that JSBSim-MAVLink integration works for UAV simulation. However, the project improves this framework by introducing backend data pipeline optimization for reducing latency, proper telemetry logging along with reliable timestamp tracking, use of time-series databases to capture high-frequency flight data and efficient post-flight analysis.

c. Reinforcement Learning Frameworks for UAV Control

(Khanzada, Maqsood, & Basit, 2025) developed an evaluation system that compares five reinforcement learning algorithms (DDPG, TD3, PPO, TRPO, SAC) for controlling fixed-wing UAVs. These algorithms were tested to determine stability of UAV flight under dynamic environmental conditions and analyzed state space parameters like velocity, altitude, orientation. By analyzing these parameters, RL agents can learn adaptability and stability strategies.

Critical Analysis: The system achieved significant performance improvement than classical PID controller with SAC achieving convergence in 400 episodes with steady-state error below 3% however, these continuous-space RL algorithms require significant computational resources and take longer to train that may converge slowly when the reward signals are weak or sparse. The paper also notes that many RL experiments use different flight configuration so comparing results across different studies becomes difficult that limits the generalizability of the findings. Most importantly, the paper mentions reward function but does not clearly describe how the rewards were designed or weighted.

Relevance to the project: This study provides strong evidence that RL can outperform traditional flight controller in dynamic environment. Our project addresses the reward design gap by implementing a structured reward formula that allows engineers to analyze how different

reward components affect learning behavior. Unlike the study by (Khanzada, Maqsood, & Basit, 2025) that focuses mainly on comparing algorithms, our project focuses on model optimization and training transparency by tracking reward component contributions, weight adjustments, convergence behavior and stability margins during training. This ensures entire optimization process not just final model performance.

d. Simulation Frameworks and Integration Architectures

(Richter, Calix, & Kim, 2024) developed comprehensive review framework of 48 research papers related to reinforcement learning for fixed-wing aircraft control. The framework categorized several important aspects of RL research including the learning algorithms used, simulation environments and tools and evaluation metrics used to measure performance. The study aimed to map overall research landscape of RL-based flight control systems.

Critical Analysis: While the system achieves valuable standardization of the research landscape, it also revealed that many researchers develop their own simulation frameworks, evaluation methods and testing environment that makes comparison of results between different studies difficult. The study also found lack of universally accepted performance metrics for important flight tasks which leads to inconsistent benchmarking. The review also highlights the lack of standardized simulation architectures as different research groups use different combinations of tools, simulators and frameworks. The study also notes that unlike quadcopter research, fixed-wing aircraft research is less developed.

Relevance to the project: This supports the need for standardization in RL learning research for UAV systems and modular simulation architecture. Our project addresses the limitation of standardization by defining a set of baseline performance metrics that can be used consistently across experiments like success rate, fuel usage, smoothness, envelope violations. Additionally, our project aims to introduce modular simulation architecture which separates different parts of simulation like flight physics simulation, visualization and environment rendering and control logic that improves transparency, reproducibility and experimental flexibility in UAV RL research.

2.3 Detailed Paper Summaries

This section presents critical analysis of 11 research papers along with the methodology and design decisions for this capstone project. Based on its methodology, strengths, limitations and direct relevance to my role of Data Architect and Model Optimization, each study is evaluated.

2.3.1 GCS Dashboard & Visualization Architecture

Summary: (Hubsky, 2024) **– GCS User Interface Analysis**

(Hubsky, 2024) developed a GCS Analysis System by using different usability evaluation criteria and review methodologies. The system evaluates the usability of drone control interfaces such as Mission Planner. The study compared different interaction approaches like keyboard-based control interfaces, eye-tracking systems and hand gesture recognition methods to improve operator efficiency and situational awareness during UAV missions.

Critical Analysis: The system focuses on visual interface design and interaction methods through thorough documentation of UI shortcomings. However, it does not prioritize the underlying data architecture need to achieve the required visual presentation. The study highlighted that Mission Planner is powerful but cluttered, but there is no exploration on improving backend data structure that could reduce interface complexity. Furthermore, the study also notes that existing GCS system cannot perform effectively in extreme operational environment like military missions as operators needs to process large amount of information under stressful conditions.

Relevance to the project: The findings of this study support the idea that both interface usability and data pipeline efficiency should be prioritized while designing GCS interface. Our project prioritizes this through structured telemetry logging, accurate timestamp synchronization and efficient data routing within the backend pipeline. Rather than only focusing on operator-facing interface, this approach ensures that telemetry data can support both real-time visualization for monitoring flight operations and analysis for model optimization. Existing GCS platforms treat data visualization and data pipeline design separately, but integrating data integrity considerations directly into system architecture addresses the gap identified in the study.

Summary: (Hansberger, Meacham, & Blakely, 2018) **– Dashboard Visualizations for UAV Systems**

(Hansberger, Meacham, & Blakely, 2018) developed a Task Analysis-Driven Visualization System that displays bullet graphs and sparkline widgets. The system redesigns UAV interfaces by analyzing how operators perform their tasks during drone missions.

Critical Analysis: The redesigned interfaced showed measurable improvements with 52% reduction in information retrieval time and 7.2% improvement in operator accuracy which demonstrates that task-focused visualization can enhance situational awareness and operator performance. Although, the system has brought improvement in visualization design and visual salience but it does not examine how that information is generated and validated within the backend system. In most modern UAV system where AI agent may participate in the control loop, data integrity requirements also become equally important. Also, there is no exploration on how emerging AI technologies could influence UAV interface design, which creates a gap between traditional operator interfaces and AI-assisted control systems.

Relevance: This study supports the concept that a GCS acts as communication hubs between human operators and automated systems. Our work builds on this concept but extends it to AI-driven UAV system by ensuring that the backend data pipeline is structured properly with tags telemetry data. Important AI-related information like decision outcomes, reward states are also logged so that the system allows dashboards to display not only aircraft's flight status but also logic behind AI decisions. This approach broadens visualization concepts by (Hansberger, Meacham, & Blakely, 2018) by ensuring visualizations are supported by transparent and trustworthy data pipeline.

Summary: (Jovanovic & Starčević, 2008)- **GCS Software Architecture**

(Jovanovic & Starčević, 2008) proposed a modular Ground Control Station (GCS) architecture that separates major UAV control functions into independent modules like flight data acquisition, real-time data analysis and command transmission using object-oriented design principles.

Critical Analysis: The modular architecture proposed in this study allows the system to visualize flight information while maintaining communication between UAV and GCS, however it has several limitations. The research is not accountable for modern developments such as AI-based autonomous control algorithms. Also, the architecture mainly focuses on basic UAV monitoring and command systems but it lacks scalability for large UAV fleets. It also lacks specific data structure required for RL training loops. Additionally, the architecture does not address user feedback mechanisms that could refine system design as per operator's experience during missions.

Relevance to the project: The study supports the importance of modular architecture in UAV control systems, which our project adopts with improvement in data structuring for AI-driven UAV systems. Our project improves this architecture by enhancing data layer through the use of high-frequency telemetry logging (60Hz or higher) required for RL training. This helps in addressing the scalability and data structure gaps identified in the research paper.

Summary: (Poma, Sojo, Maza, & Ollero, 2025)- **Multi-UAV Ground Control Station**

(Poma, Sojo, Maza, & Ollero, 2025) developed an Open-Source Multi-UAV GCS for managing multiple UAVs simultaneously. The system performs automated workflows for mission operations like mission planning, data collection, data processing and also integrate Traveling Salesman Problem (TSP) for optimizing flight paths. This helps in efficient operations for multi-UAV fleets performing inspection tasks.

Critical Analysis: While the system provides strong scalability and flexibility for UAV operations, it is dependent on specific autopilot system. This restricts its use on UAVs that use different flight control hardware or software. In addition, the study focuses on mission logistics and operational workflow, rather than structure and fidelity of telemetry data used for machine learning or model training. Also, there is lack of external integration capabilities and retrieval of detailed mission logs for advanced data analysis.

Relevance to the project: This study supports the role of GCS as an intermediary system for coordinating UAV operations, which is focused on our project with improvements in data pipeline flexibility. Our project adopts their workflow automation but implements application-agnostic telemetry parsing and structured data logging. This ensures flexibility and compatibility across different UAV platforms and control systems.

Summary: (Ufelek, Yoruk, & Cakici, 2020)**- Real-Time Telemetry Visualization**

(Ufelek, Yoruk, & Cakici, 2020) developed a real-time telemetry monitoring system for flight testing. This system was built using Time-Series Database (TSDB). IT performs data acquisition, real-time visualization and even anomaly detection during UAV flight tests.

Critical Analysis: While the system improves data storage and retrieval speed with the use of TSDB, the research lacks exploration on specific machine learning algorithms used for anomaly detection in detail. Also, there is no proper information on integration challenges between real-time monitoring software and simulation environments used in AI training systems. The study highlights how thousands of telemetry parameters are collected by modern UAV system in very high frequencies which creates challenges in identifying most relevant signals for ML models.

Relevance to the project: The study supports the importance of Time-Series Databases for managing high-frequency telemetry data. Our project builds on this idea by adopting TSDB architecture optimized specifically for RL workflows. As, data architect, I prioritize their database approach but enhance it to improve database query performance for extracting training episodes which includes system states, agent actions and reward values. This helps in efficient retrieval of structured telemetry data for RL training and evaluation.

2.3.2 Telemetry Data Pipeline & Logging

Summary: (Sivamani & Gudipalli, 2024)**- UAV Data Logging System**

(Sivamani & Gudipalli, 2024) developed a UAV Data logging System that uses MEMS-based IMU sensors with GPS module and Mahony filter to combine IMU and GPS data and improve orientation. This system evaluates different data logging formats, by comparing CSV and binary based log to identify which methods provide better efficiency and performance to record UAV telemetry data.

Critical Analysis: While the system demonstrates successful sensor fusion using Mahony filter, the system is designed only for low-altitude UAV application which limits its applicability in complex scenarios. Also, the study also identifies potential data corruption risks when UAVs move outside radio transmission range that can lead to incomplete telemetry data. Although the study has compared logging format, there is no exploration on impact of log structure in machine learning workflows. Additionally, there are many stability challenges experienced in Mahony filter when estimating pitch, roll and yaw during abnormal conditions that can make logged telemetry data unreliability.

Relevance to the project: The study supports the importance of logging architecture and data formats in UAV telemetry systems. This concept is used in our project and there is more extending into logging architecture to support structured schemas optimized for RL workflows. I adopt their modular logging system but extend it to introduce timestamp synchronization mechanisms, structured telemetry schemas and reward component logging for RL agents. This not only help in monitoring but also efficient downstream model training and optimization.

Summary: (Aziz & Loya, 2025)- **AeroQT MAVLink Test Bench**

(Aziz & Loya, 2025) developed AeroQT which is a lightweight GCS designed for UAV simulation experiments that integrates JSBSim and MAVLink using UDP and TCP communication protocols. It performs software-in-the-loop simulations, that helps in test flight control systems and evaluate aircraft parameters in a simulated environment before deploying them in real aircraft.

Critical Analysis: While AeroQT shows successful integration of flight dynamics with ground control, there is telemetry latency of approximately 400ms which affects real-time controller bandwidth. The architecture only focuses on single-aircraft testing scenarios so it does not support multi-UAV operations or advanced mission planning capabilities. Furthermore, there is lack of advanced visualization and monitoring tools which could help enhance situational awareness during simulation experiments.

Relevance to the project: This study supports the concept of JSBSim-MAVLink integration is effective framework for UAV simulation research. Our project adopts this intergration approach and furthermore improves it with efficient backend telemetry pipeline to avoid latency. 400ms latency identified by (Aziz & Loya, 2025) is addressed in our project by implementation of structured data ingestion layers and timestamp synchronization mechanisms. This ensure data accuracy reflecting simulation state in real time and also reduces the risk that delayed telemetry will cause with control loops and RL training.

Summary: (Defense Technical Information Center, 2022)- **Telemetry Data Mining for UAS**

(Defense Technical Information Center, 2022) developed a Telemetry Data Mining System that uses supervised learning methos lie CART decision trees, C5.0 classification algorithms and neural networks to classify phases of flight such as takeoff, cruise and landing based on telemetry variables recorded during operations.

Critical Analysis: While, there is successful classification of flight phases, data quality issues are seen as the source logs contained null values in "Prop_P_Pi" variable which shows inadequate validation mechanisms. Even the dataset used is relatively small and static which limits the robustness of analysis. Furthermore, there was lack of exploration of other analytical methods as only three machine learning algorithms were evaluated.

Relevance to the project: This project supports the concept that telemetry data contain hidden analytical value within them. Our project supports this concept with improvements in real-time validation which addresses the data quality gap identified by DTIC. This gap is addressed by implementing validation layer that flags corrupted or null value before entering the database. This approach ensures that machine learning systems are operated on high-integrity telemetry streams and are reliable for both monitoring and RL analysis.

2.3.3 Model Optimization and Reward Function

Summary: (Khanzada, Maqsood, & Basit, 2025)- **RL for Fixed-Wing UAV Flight Controls**

(Khanzada, Maqsood, & Basit, 2025) developed a Comparative RL Evaluation System using continuous-space algorithms (SAC, PPO, DDPG, TD3, TRPO) which were tested in adaptive control environments to analyze different state-space parameters influencing learning performance.

Critical Analysis: While the study demonstrates high performance improvement in comparison to PID controllers where SAC model achieved convergence in 400 episodes, there was no clear documentation on assignment of optimization weights to different performance objectives. There was lack of specification on reward function design and how stability metrics were balanced against task completion performance. Furthermore, continuous action-space reinforcement learning models like the ones used in this paper has several challenges like higher computational requirements and slower convergence in environment with sparse rewards which make it difficult to evaluate the achieved performance improvements reported.

Relevance to the project: This study supports the concept that reinforcement learning approaches can outperform traditional control strategies in adaptive flight control tasks. Our project builds on this finding by introducing documentation of reward function which helps in analyzing how individual reward components influence agent behavior.

Summary: (Richter, Calix, & Kim, 2024) **– Review of RL for Fixed-Wing Aircraft**

(Richter, Calix, & Kim, 2024) developed a Comprehensive Review Framework by systematically examining 48 research papers and categorizing them. It performs systematic analyzing of RL methodologies, simulation environments, evaluation metrics and control objectives which helped in identifying major trends and methodological patterns.

Critical Analysis: While the study provides valuable overview of research landscape, the study concludes that no universally accepted evaluation metrics currently exists for UAV control tasks. Many researchers develop their own toolkits, simulators and performance metrics so comparing result across different studies is difficult.

Relevance to the project: This supports the need of standardized metrics and modular architecture is needed for meaningful RL research. Our project addresses this gap by implementing documented benchmarking metrics like mission success rate, fuel efficiency, control smoothness, flight envelope violations. This enables other researchers to compare their RL models against consistent performance benchmarks.

Summary: (Tovarnov & Bykov, 2022) **– RL Reward Function for UAV Control**

(Tovarnov & Bykov, 2022) developed a Reward Function design approach for UAV reinforcement learning systems. It performs validation in virtual simulation environment across several UAV task scenarios.

Critical Analysis: (Tovarnov & Bykov, 2022) proposed a function based on estimating flight time using third-order Bezier curves. They proposed a reward function that includes three parts:

$$R\ (s, a) = w_1 R_{task} + w_2 R_{smoothness} + w_3 R_{safety}$$

Each part represents a specific objective, where Task reward encourages the drone to reach the target efficiently, smoothness reward encourages smooth flight control by penalizing abrupt control surface changes, safety reward ensures the drone stays within safe flight limits. The validation experiments were done in simplified two-dimensional environment which does not fully represent the complexity of real UAV flight dynamics. Furthermore, the study also does not fully examine how reward weights interact with three dimensional aerodynamic constraints like stall conditions, spin recovery and aerodynamics load limits.

Relevance to the project: This project supports the concept that trajectory-based reward shaping can significantly improve RL performance. The reward function designed in this paper will be implemented in our project as it analyzes how individual reward components influence agent behavior but there will also be inclusion of three-dimensional flight envelope parameters like angle of attack limits, G-force thresholds. Implementation of these safety constraints in reward formula, will help RL agent to remain safe withing realistic aerodynamic conditions. This addresses the limitation of reward function being optimized only in simplified simulation environments may fail when applied to real-world UAV operations.

2.4 Methodology, Technology, and Tools Analysis

This section represents the key technology choices derived from the literature review along with evaluation based on strengths, limitation and align with my role as Data Architect and Model Optimization.

2.4.1 Data Pipeline Architecture: Storage and Structure

Evaluation: How UAV telemetry data is stored and organized is very important especially for reinforcement learning systems that require high-frequency telemetry (like 60Hz or more). (Ufelek, Yoruk, & Cakici, 2020) used a Time-Series Database (TSDB) for telemetry storage which is very fast for time-based data like telemetry data, yet noted challenges in analyzing thousands of parameters collected at very high frequency. Similarly, (Sivamani & Gudipalli, 2024) compared CSV and binary logging, finding that CSV is computationally efficient for simple application but CSV alone is not reliable for long missions. (Defense Technical Information Center, 2022) also noted the serious data quality issues which emphasizes the importance of data validation before storing them.

Justification: As a data architect in this project, I will adopt Ufelek's Time-Series Database approach for high-frequency telemetry as it is necessary for fast access to training data but use Shivamani's modular logging concept by using CSV/JSON for offline analysis as it is human readable. Our pipeline will also include a validation layer so that the system checks missing values, corrupted packets and malformed telemetry messages before they enter the database which prevents problems mentioned in DTIC report.

Table 1: Data Storage Selection

Component	Selection	Justification
Primary Storage	Time-Series Database	Used this for optimizing for high-frequency telemetry data that is required for RL training
Backup Format	CSV/JSON	Use of CSV as it is human readable and using for

		offline debug and post-flight analysis
Data Validation	Pre-Ingestion Checks	Checking data integrity like missing values before it gets to database.
Schema Design	Structured Tags	For efficient querying of state-action reward episodes for RL training

2.4.2 Telemetry Protocol: MAVLink & Transport Layer

Evaluation: To communicate between system components like JSBSim and GCS, communication protocol like MAVLink is commonly used but it also needs transport layer i.e. UDP and TCP. (Aziz & Loya, 2025) developed AeroQT using MAVLink with UDP/TCP protocols but showed 400ms latency that reduces controller responsiveness. (Jovanovic & Starčević, 2008) emphasized on Quality of Service (QoS) but that system did not support modern AI pipelines. Similarly, (Poma, Sojo, Maza, & Ollero, 2025) noted that many GCS system were tied to specific autopilot hardware making it less flexible to switch UAV platforms.

Justification: This supports the concepts that MAVLink is important with improvements in backend data pipeline which our project will incorporate. For addressing the latency problem, our project will use structured data ingestion layers with timestamp synchronization mechanisms which allows the system to align telemetry, simulation state and AI decisions. We will also prioritize UDP for high-frequency telemetry while TCP for critical command messages balancing latency and reliability trade-off better than single protocol approach.

Table 2: Telemetry Protocol Selection

Component	**Selection**	**Justification**
Message Protocol	MAVLink	Standard UAV communication protocol
Transport Layer	UPD/TCP	UDP helps in fast streaming of sensor data, TCP for reliable control messages
Latency Mitigation	Timestamp Synchronization	Reduces telemetry delay
Flexibility	Autopilot-Agnostic	Works with multiple UAV platforms

2.4.3 Reward Function Design: Continuous vs Sparse

Evaluation: Design of reward function significantly influence the stability of AI co-pilot. There are two main types of rewards used in RL i.e., sparse rewards (reward only after task is completed) which led to slow convergence, and continuous rewards (small rewards received continuously during the task) which accelerate learning but risk reward hacking. (Khanzada, Maqsood, & Basit, 2025) compared several RL algorithms which outperformed PID controllers but sparse rewards caused slow convergence. (Tovarnov & Bykov, 2022) proposed Bezier trajectory reward shaping but it was tested in 2D simulation only. Similarly, (Richter, Calix, & Kim, 2024) noted that reward function often works in simulations but fail in real aircraft as safety penalties are too weak to handle real complex situations.

Justification: Our project will be using structured reward function: $R(s,a) = w_1 R_{task} + w_2 R_{smoothness} + w_1 R_{safety}$ where each component is logged separately for post-flight analysis. Unlike Tovarnov's 2D validation, we will be using 3D envelope protection metrics like angle of attack, G-Force which addresses Richter's concern about sim-to-real failure

Table 3: Reward Function Design Choices

Component	Selection	Justification
Reward Type	Dense (Continuous)	Compared to sparse reward, it accelerates convergence, enhance fast learning
Safety Metrics	Envelope Protection	Prevent dangerous flight that works in sim but fails in reality
Structure	Weighted Sum (w1, w2, w3)	Balance objectives, allows tuning of task vs. safety priorities during optimization
Logging	Component-wise	Analyze RL decisions, enables post-flight analysis

2.5 Summary of Key Findings and Research Gaps

This section synthesizes key findings from reviewed studies along with gaps in existing research connecting reviewed work to current project and demonstrating how past research has informed the proposed approach.

2.5.1 Key Findings

After critical analysis of 11 papers, the following key findings are highlighted that justifies the design decisions in the project:

a. **RL Outperforms Classical Control:** Traditional flight control systems uses classical controllers like PID which works well in only simple and predictable environment so RL algorithms like SAC and DDPG performed better than PID controller showing improvement in flight stability, adaptability and disturbance recovery (Khanzada, Maqsood, & Basit, 2025).
b. **Fidelity Matters:** Low-fidelity simulators simplify physics calculation. (Richter, Calix, & Kim, 2024) found that RL agents trained in low-fidelity simulations often fails in more realistic environment as agent only learns how to work on simplified simulator.
c. **Data Quality is Critical:** Sometimes, telemetry datasets also contain null value, invalid sensor readings and corrupted telemetry packets as discovered by (Defense Technical Information Center, 2022). Poor data quality led to unreliable flight analysis and incorrect machine learning model.
d. **Latency Bottlenecks:** While JSBSim-MAVLink integration is viable, (Aziz & Loya, 2025) found 400ms latency which affects controller bandwidth and in real-time control systems even small delays can reduce stability and affect AI decision making.
e. **Reward Transparency:** Most studies only report final performance results of the model without explaining how reward functions are designed and how different reward components influence learning. (Khanzada, Maqsood, & Basit, 2025) (Tovarnov & Bykov, 2022).
f. **UI vs. Data Architecture:** Most GCS research focuses on improving user interface and making dashboard readable but underlaying data pipeline to achieve usability goals are ignored which creates disconnection between data collection and data visualization. (Hubsky, 2024)

2.5.2 Identified Research Gaps

After the synthesis of reviewed literature, several research gaps and limitations are identified which justify the proposed capstone work:

a. **Lack of integrated data schemas for RL:** Different studies focus on different parts of the UAV system. While (Sivamani & Gudipalli, 2024) focused on data logging and telemetry systems and (Khanzada, Maqsood, & Basit, 2025) focused on RL reward function, no study combines these two properly. Lack of unified data schema that store reward function components together with telemetry data brings post-flight model optimization difficulties because when engineers analyze flight logs later, they cannot easily connect telemetry data and reward penalties given to the AI.

b. **Telemetry Latency in Simulation Bridges:** (Aziz & Loya, 2025) found that communication between JSBSim simulator and MAVLink protocol creates approximately 400ms of delay. However, existing literature only identifies the delay but do not propose backend solutions to reduce it. This shows a gap between backend optimization strategies to reduce latency in telemetry pipelines.

c. **Lack of standard safety metrics for AI pilots:** Lack of standardized safety metrics is highlighted in study by (Richter, Calix, & Kim, 2024). Most of the studies evaluate success using navigation accuracy and task completion while ignoring important safety factors such as: flight stability, envelope protection, safe maneuver limits which leads to AI agent completing task successfully but still fly dangerously. Hence, there is currently no standardized framework for safety metrics in UAV reinforcement learning.

d. **Lack of Analytical dashboard for AI monitoring:** Researches on GCS done by (Hubsky, 2024) and (Hansberger, Meacham, & Blakely, 2018) focused on operator UI but there is limited research on AI monitoring dashboard designed for data architects to monitor reward convergence, training loss curves, AI stability metrics alongside flight data.

e. **Poor dataset quality in telemetry research:** Another issue discovered in telemetry research is data quality. (Defense Technical Information Center, 2022) discovered telemetry dataset having missing values, incomplete sensor readings and static datasets that are not updated. There is a gap in real-time data validation in telemetry pipelines to detect missing value, validate the incoming data and ensure data integrity before using them for AI training.

Table 4: Observed Limitations, Research Gap, and Impact on Proposed Project

Authors	Year	Observed Limitation	Resulting Research Gap	Impact on Project
Richter et al.	2024	Lack of standard safety metrics for RL	Absence of framework that teaches AI systems to prioritize stability during learning	Addressing this by creating custom stability metrics in reward function
Aziz and Loya	2025	400ms telemetry delay during connection between JSBSim and MAVLink.	Lack of backend optimization strategies in real-time telemetry logging.	Improving data pipeline architecture, optimizing timestamp synchronization to improve logging speed.

Hubsky	2024	GCS interfaces are clustered and outdated.	Lack of dashboard containing both operational control and analytical dashboard for AI monitoring.	Emphasize the need for clean and modular analytical dashboard.
DTIC Report	2022	Missing value and static telemetry dataset.	Lack of strong data validation mechanisms	Requires data validation mechanisms inside logging pipelines
Khanzada et al.	2025	Sparse reward system cause instability in RL environment	Lack of reward function that continuously guide AI behavior during flight	Support implementation of continuous reward shaping.

2.6 Summary Table of Past Works

Table 5: Summary of Past works

Authors	**Year**	**Methodology**	**Key findings**	**Contribution**
Hubsky	2024	Analysis of existing GCS interfaces like Mission Planner and their usability for drone operation.	Existing interfaces are powerful, but contains too many panels causing information overload and confusion.	Highlighted importance of user-centered interface design that reduce cognitive load in operators.
Hansberger et al.	2018	Redesigned UAV UI using task analysis, introducing tools like bullet graphs, sparklines.	Redesigned interface improved operator accuracy by 7.2% and information retrieval speed by 52%.	Showed that task specific visualization improves operator performance and creates awareness about situation.
Jovanovic & Starcevic	2008	Designed a software architecture for UAV systems that collects and analyzes flight data in real time.	Proposed modular architecture where different component handles data acquisition, data visualization, mission monitoring.	Established a foundational software architecture, informs the decision to decouple backend data pipeline from frontend dashboard.
Poma et al.	2025	Developed an open-source GCS that can manage multiple UAV-fleets simultaneously, with TSP-based planning algorithm	System was successfully validated in real world scenarios, showing improved automation and operational flexibility.	Introduced a general-purpose GCS framework that can manage automated workflow from task assignment to data delivery.
Sivamani & Gudipalli	2024	Built a data logging system using IMU and GPS sensor and for combining sensor data used Mahony filter.	Mahony filter provided stable and computationally efficient sensor fusion that improved flight	Demonstrated low cost sensors combination with efficient filtering algorithm that can create reliable

			tracking accuracy.	telemetry systems.
Aziz & Loya	2025	Developed AeroQT test environment that integrates JSBSim & MAVLink that sends simulation data using UDP/TCP.	400ms telemetry delay was observed that can reduce performance of real-time control systems.	Their framework enabled software-in-the-loop simulations and real-flight testing validation.
Ufelek et al.	2020	Built software that monitors real-time telemetry data and uses Time-Series Database for storing data.	As database expanded, ML algorithms improved prediction accuracy.	Emphasized the importance of real-time visualization and Time-Series Databases in analyzing flight tests.
Khanzada et al.	2025	Compared 5 RL algorithm for UAV control (DDPG, TD3, PPO, TRPO, SAC).	SAC performed best by achieving convergence in 400 episodes with <3% steady-state error.	Showed that continuous-space RL algorithm can outperform traditional PID controllers.
Richter et al.	2024	Comprehensive meta-analysis of 48 papers on RL for aircraft control,	Discovered lack of standardized safety metrics, reliable sim-to-real transfer methods.	Offered comprehensive overview of RL methodologies and highlighted major research gaps in AI safety evaluation and benchmarking for RL.
DTIC Report	2022	Use of ML methods (CART, C5.0, neural networks) to analyze telemetry datasets.	Discovered that abnormal drone behavior can be detected by electrical current draw, but dataset had missing values that limited the analysis.	Demonstrated the potential of ML for uncovering hidden value in telemetry data.
Tovarnov, M.S., & Bykov	2022	Used deep RL with new reward function based on Bezier curve trajectory estimation.	Reward function validated in virtual environment, addressing navigation tasks and interception tasks.	Introduced a new reward shaping technique based on trajectory prediction.

Conclusion

Synthesized analysis of literature establishes a clear foundation for data architecture and model optimization proposed in this project. Existing researches have shown consistent pattern across four major themes: GCS interface design, telemetry data logging, RL reward functions and simulation pipelines. However, these components are studied separately and existing research falls short in combining them in one integrated system. (Jovanovic & Starčević, 2008) and (Poma, Sojo, Maza, & Ollero, 2025) demonstrated the importance of modular system

architecture to improve flexibility and scalability, while (Hubsky, 2024) and (Hansberger, Meacham, & Blakely, 2018) shows that many existing interfaces contain too much information, are clustered and difficult to interpret so, operators cannot easily understand why an AI system makes certain decisions. (Sivamani & Gudipalli, 2024) and (Defense Technical Information Center, 2022) shows that logging reliable UAV data is more complex than it appears. (Khanzada, Maqsood, & Basit, 2025), (Richter, Calix, & Kim, 2024), (Tovarnov & Bykov, 2022) collectively show that structured logging of reward components is required for better model optimization.

Among the limitations found in the literature, two gaps that are especially important for this project are: no unified data schema for AI training and telemetry latency in simulation environment. Most systems log telemetry data and sensor readings but do not store RL reward components together with telemetry data and study by (Aziz & Loya, 2025) identified 400ms latency which threatens timestamp integrity across the pipeline. These gaps form the core of the backend design suggested within this project where there is structured metadata logging, real-time data validation and separation of high-frequency AI data and lower-frequency mission data. By solving these gaps, the project not only support the AI co-pilot system developed in this capstone but also contribute to provide a general data architecture framework that future UAV simulation system can use.

CHAPTER 3: SYSTEM ANALYSIS

3.1 SDLC Methodology: SCRUM Framework

In this section adoption of SDLC methodology is outlined where SCRUM Agile framework is selected. This methodology is selected to accommodate iterative nature of research-based development and align with Capstone 1 (Research and Design)

3.1.1 SCRUM Roles

Table 6: SCRUM Roles

Role	Team Members	Responsibilities
Product Owner	Supervisor	Sets project vision, approves research scope and validates design and mockups
SCRUM Master	Saugat Chaudhary Tharu (Team Lead)	Manages sprint backlog for design tasks, removes blockers, keeps the team updated
Development Team	All 5 members	Executes sprint tasks
Data Architect	Rija Bhomi	Owns data pipeline and GCS wireframe tasks, telemetry logging schema and reward function research

3.1.2 Sprint Structure

The project is divided into 4 sprints.

Table 7: Sprint Structure

Sprint	Duration	Duration (Weeks)	Focus
Sprint 0	Jan 7- Feb 18	6 weeks	Preparation & Proposal
Sprint 1	Feb 19- Mar 10	3 weeks	Research & Literature
Sprint 2	Mar 11- Mar 24	2 weeks	System Design & Architecture
Sprint 3	Mar 25- Apr 3	1.5 weeks	Finalization & Submission

Detailed Sprint Breakdown

Table 8: Detailed Sprint Breakdown

Sprint	Task	Start Date	End Date	Duration
Sprint 0	Group formation & Topic Finalization	7-Jan	30-Jan	23
	Requirement gathering	1-Feb	7-Feb	6
	Initial Proposal Defense	8-Feb	10-Feb	2
	Final Project Proposal Submission	3-Feb	10-Feb	7
	Supervisor Allocation	18-Feb	19-Feb	1
Sprint 1	Define Flight Data Points & Telemetry	19-Feb	26-Feb	7
	Literature Review	27-Feb	20-Mar	21
	Initial Wireframe concept (GCS Dashboard)	19-Feb	20-Mar	29
	Technology Stack Research	19-Feb	3-Mar	12
Sprint 2	Data Architecture Design	11-Mar	20-Mar	9
	UML Diagrams (Use case, activity)	12-Mar	20-Mar	8
	System Design (Chapter 4)	18-Mar	28-Mar	10
	Sequence Diagrams	20-Mar	26-Mar	6
	System Analysis (Chapter 3)	20-Mar	29-Mar	9
Sprint 3	SCRUM Plan & Gantt	26-Mar	30-Mar	4
	Mockup Prototype	20-Mar	1-Apr	12
	Report Formatting and Polish	27-Mar	2-Apr	6
	Cross-Chapter consistency check	29-Mar	31-Mar	2
	Final review and proofreading	1-Apr	2-Apr	1
	Final Submission	3-Apr	3-Apr	0

Milestone Tracking

Table 9: Milestone Tracking

Milestone Name	Target Date	Status	Deliverable	Owner
Project Proposal Approved	10-Feb	Complete	Approved Proposal	All
Supervisor Allocated	19-Feb	Complete	Supervisor Confirmation	All
Literature Review Complete	10-Mar	Complete	Chapter 2 draft	All
Initial Wireframes Approved	5-Mar	Complete	Low-Fi Wireframe	Shrutika, Rija
System Design complete	24-Mar	In progress	Chapter 3 & 4	All
All UML diagrams complete	22-Mar	Complete	Use Case, activity, sequence diagram	All
Report draft complete	31-Mar	In progress	Complete draft	All
Final mockup ready	1-Apr	In progress	High-FI prototype	Shrutika, Rija
Final submission	3-Apr	Pending	Submitted Report	All

Gantt chart in appendix

3.2 Theoretical: SWOT Analysis

This section provides strategic evaluation of preferred frameworks, tools, programming language and architecture for this capstone project. Each technology is evaluated using SWOT (Strengths, Weaknesses, Opportunities, Threats) to justify the selection of technology stack based on literature findings from Chapter 2.

3.2.1 Programming Languages:

The data pipeline and reward function require a language that balance between high-speed execution needed for physics and flexibility for RL training.

Table 10: SWOT analysis of Programming Languages

Features	Python (RL/Backend)	C++ (Physics/GCS)	GDScript (Godot UI)
Strengths	Industry standard with ML rich libraries like PyTorch, Stable Baselines3 which helps in rapid prototyping.	JSBSim native language, High performance speed for real-time execution Low-level memory control.	Easy to make 3D visualization and optimized for Godot
Weakness	Compared to compiled languages like C++,	Slower development time and steep learning	Limited to Godot environment only.

	python has slower execution speed.	curve	
Opportunities	Smooth and easy integration with Godot via wrappers.	Direct native integration with JSBSim Can optimizie telemetry parsing for sub-milisecond	Very suitable for real-time telemetry visualization.
Threats	Version compatibility issues between ML libraries Global Interpreter Lock can limit multi-threading	Harder to debug Memory management complexity issues and memory leak if not managed properly.	Less community support than Python.

Hence, a hybrid approach is selected where GDScript will be used for dashboard and real-time visualization in HUD and GCS, Python will be used for model optimization and RL training to ensure seamless integration with JSBSim and Godot engine.

3.2.2 Data Storage

To ensure data integrity for RL training, telemetry data must be stored accurately so that even high frequency data is properly stored and missing and corrupted data are flagged.

Table 11: SWOT analysis of Data Storage

Features	**InfluxDB (Time-series)**	**CSV/JSON files**	**PostgreSQL**
Strengths	Optimized for high-frequency data streams Also has fast data retrieval speed suitable for RL training	They are human readable with no database setup required that can be used for offline debugging and post flight analysis	Offers ACID compliance, strong data consistency and support complex relational queries
Weaknesses	Uses a separate query language called Flux so learning it can slow development initially	CSV are not scalable for large datasets causing slow search operations, inefficient queries.	Can be slow for high-frequency time-series telemetry data
Opportunities	Has powerful capabilities for real-time telemetry dashboards, live flight monitoring Keeps storage efficient while preserving historical trends.	CSV can be used to analyze telemetry in Python and import data into machine learning pipelines.	Can store processed telemetry datasets after validating and filtering them
Threats	Can introduce learning curve Requires high maintenance and a running database server	If there is connection loss, then file may have null values, incomplete data and such corrupted files can cause error while RL training.	Performance bottlenecks and slow down RL training pipelines

For this project, CSV/JSON format is selected for primary data storage as minimal write-latency is prioritized. InfluxDB is widely used in industrial fleets but as they require database server it can introduce jitter in telemetry stream. So, using JSON and SCV format for data storage will be

flexible as they can store both raw telemetry data and AI reward states in lightweight file. This is also natively compatible with both Godot and python's machine learning libraries.

3.2.3 Communication Protocols

For telemetry transmission between JSBSim and data pipeline, communication protocol is a backbone that ensures speed and reliability.

Table 12: SWOT analysis of Communication Protocols

Features	MAVLink	JSON+ WebSockets
Strength	It is efficient and uses binary format so low bandwidth usage and low latency communication Also, it is standard protocol in UAV ecosystems	Text based data format so easy for humans to read and debug
Weakness	Not as simple to work with as JSON as developers need to parse MAVLink headers, decode message structures and use specific libraries	Extremely simple and flexible and when combined with WebSockets, JSON can support real-time communication between systems
Opportunities	Helps in smooth transition from Software-in-the-loop to Hardware-in-the-loop	Rapid prototyping and good for initial UI testing
Threats	Version conflict as cause communication breakdown	In large data, jitter can be caused which can lead to lagging in the 3D visualization environment.

MAVLink is selected as primary communication protocol for this project. This decision is driven by performance requirement as JSON can be easier to debug but it wastes significant bandwidth and increases processing latency. While MAVLink has binary format so data is packed into few bytes. It ensures that the 60Hz telemetry refresh rate target is met without taxing CPU. Also, MAVLink ensures data integrity so it guarantees that RL agent receives perfectly synchronized state data from JSBSim.

3.2.4 Frameworks

The choice of game engine determines how telemetry is rendered.

Table 13: SWOT analysis of Frameworks used

Features	Godot Engine	Unity Engine
Strength	Open-source architecture which allows custom integration and lightweight	Very large ecosystem with massive asset store and pre built components and supports high quality 3D rendering
Weakness	Smaller ecosystem compared to Unity so developer may need to build more components manually	Heavy and causes software overhead Its proprietary software so deep modification is limited
Opportunities	As it is open source, it allows deep customization making it flexible for the project	It's strong rendering capabilities help in creating realistic digital twin environment that closely

		match real-world behavior
Threat	Lack of pre-built flight simulation templates	Higher hardware requirements Licensing restrictions for some use cases.

Godot is chosen in this project as its open-source which allows deep customization which makes it possible to implement custom MAVLink telemetry nodes, real-time telemetry visualization and specialized dashboards. Although Unity is more suitable for large-scale commercial simulation environment, it introduces unnecessary complexity for GCS interface. While Godot is lightweight, flexible and ensures 60Hz telemetry refresh rates without proprietary software bottlenecks.

3.2.5 ML Framework

In reinforcement learning systems, ML framework is very important architectural decision as it dictates the speed of convergence, reproducibility of results and visualize decision-making logic.

Table 14: SWOT analysis of ML Framework

Features	**Stable Baselines3**	**Custom Implementation**
Strengths	Popular open-source reinforcement learning library built on PyTorch and provides pre-implemented algorithms like DQN, PPO, SAC	Completely flexible as it is customizable allowing optimization for aerodynamic limits and flight envelope constraints.
Weakness	Limited customization for complex reward wrappers due to some black-box elements	Building RL algorithms from scratch is very complex leading to steep learning curve and slow development.
Opportunities	Allows for Observation Wrappers which help to extend algorithms with custom reward wrappers enabling transparency and component-wise reward logging.	Custom framework like Angle of Attack and G-Force limits could produce more specialized RL controllers.
Threats	Version compatibility issues, may not support multi-agent extensions later	Without standardized implementation, it becomes difficult to compare results with existing papers.

Stable Baselines3 framework is selected as it ensures standardization and reproducibility. This will help in making sure that the RL algorithms follow established implementation where results can be compared with existing studies too.

3.3 Technical Analysis

SWOT analysis have justified the selection of tools and this section presents the technical choices which is derived by evaluating based on strengths and limitation with its justification to meet high-frequency requirements of flight simulation GCS.

3.3.1 Libraries and Tools

Libraries are selected on the basis of ML ecosystem support, JSBSim compatibility and Godot integration.

Table 15: Libraries and Tools used

Technology/Tools	Component	Justification
Python 3.9+	Backend	Python will be used as primary programming language for telemetry logging and ML algorithms and reward function design as it is compatible with ML libraries and easy for integrating Godot's Python API.
PyTorch	Deep Learning Framework	PyTorch provides required libraries to build DQN model offering dynamic computation graphs and reward function experimentation.
Stable Baselines3	RL algorithm library	Pre-implemented RL algorithm that ensures standardization and reproducibility which will prevent steep learning curve
JSBSim 1.x	Flight Dynamics Engine	Open-source flight dynamics model that provides high-fidelity 6-DoF physics for aircraft simulation and gives simulation that is close to real-world aircraft
Godot 4.6	Visualization engine	It is lightweight and open-source 3D game engine with seamless 3D UI integration for GCS dashboard
GDScript	UI Scripting	Similar to python syntax so easy to learn and ensure rapid prototyping. Also it is natively developed for Godot
TensorBoard	Monitoring and Visualization	It provides visualization for AI training, shows reward convergence and loss curves along with model performance metrics in real-time.
Git/GitHub	Version Control	Git/GitHub will be used to maintain collaborative workflow, and to ensure code backup.

3.3.2 GCS Interface & Telemetry Mapping

GCS is not only being used for visualization but in this project it is also designed as Diagnostic Data Tool that is used to visualize flight behavior, monitor AI decisions and analyze telemetry data.

Table 16: GCS interface and Telemetry mapping

GCS Widget	Mapped Telemetry	Refresh Rate	Data Pipeline Path
Primary Flight Display (Artificial Horizon)	pitch, roll	60 Hz	UDP Socket – Godot Engine - Shaders
AI Confidence Bar	Reward value, policy entropy	10 Hz	RL agent – JSON – Godot UI
Flight Path Vector	Groundspeed, climb rate	30 Hz	JSBSim Simulator – MAVLink – GCS
Log status indicator	FILE_TRANSFER_PROTOCOL	1 Hz	Telemetry pipeline – CSV

			writer

GCS architecture seperates telemetry into two categories of data. High speed data that uses UDP as it provides extremely low latency and minimal CPU overhead and ensure UI responsiveness. Next is lower-speed analytical data which use JSON as it is easier to debug and simpler for data parsing. This architecture ensures performance and scalability of GCS interface.

3.3.3 Reward Function Implementation

To ensure the RL agent convergence and safety, reward function design plays a significant role. Dense, safety-weighted rewards are needed to ensure stability in UAV systems. Our system will incorporate a structured reward function.

$$R\ (s, a) = w_1 R_{task} + w_2 R_{smoothness} + w_3 R_{safety}$$

Table 17: Reward Function Implementation

Component	Selection	Justification
Reward Type	Dense (continuous)	Continuous feedback at every step will help the agent to learn much faster with faster convergence during training
Safety Metrics	Angle of Attack (AoA), G-Force limits	These constraints help in sim-to-real transformation as it helps the agent to learn realistic and physically safe flight behaviors.
Structure	Weighted sum (w1, w2,w3….)	"R_task" will measure task performance, "R_smooth" will encourage smooth control inputs and "R_safety" will penalize unsafe behavior", this structure will allow transparent tuning of reward priorities.
Logging	Component-wise	Helps to understand how the reward is generated during training and allow post-flight reward analysis that will help to make learning process easier to understand and evaluate.

3.4 Requirement Specifications

This section outlines the functional and non-functional requirement of the project derived from critical gaps identified in Chapter 2 (Literature Review) to ensure the system promotes data integrity, reward transparency and addresses latency issues.

3.4.1 Functional Requirements

It defines the specific behaviors and function that a system must have to function data pipelining, logging, GCS and reward calculation.

Table 18: Functional Requirements

ID	Requirement Name	Description
FR-01	Telemetry extraction	The system must be able to extract 6-DOF flight data from JSBSim simulator using MAVLink telemetry packets.
FR-02	Asynchronous Parsing	The data pipeline must use asynchronous processing (packet reception, packet decoding, data storage) to occur in parallel with JSBSim so that it does not block 60Hz simulation physics loop.

FR-03	State Normalization	Before the telemetry data is sent to RL training, it must be rescaled to range -1 to 1 to ensure faster convergence and stable ML training.
FR-04	Multi-Objective reward	Reward function must calculate each component (R_task, R_smooth, R_safety) separately and combine them into total weighted reward
FR-05	Component-wise-logging	Every component of the system must be stored into CSV/JSON file separately to ensure explain ability
FR-06	Real-time HUD Sync	GCS must rapidly update the artificial horizon and other gauges with telemetry at 60 frames per second to display smooth real-time movement.
FR-07	Control Authority Switch	The system must allow human pilot to override AI control immediately.
FR-08	Reward Computation Time	Reward function calculation must be completed within 10ms to maintain real-time control responsiveness
FR-09	Anomaly Alerting	System must trigger a visual warning in the GCS when unsafe flight conditions occur.
FR-10	Historical Log Replay	The system must allow the admin/developers to review previous flight session inside GCS dashboard.

3.4.2 Non-functional Requirements

These requirement helps to ensure the performance, reliability and data integrity in a system.

Table 19: Non-functional Requirements

ID	Requirement Name	Description
NFR-01	Telemetry Latency	Maintaining below 100ms latency to prevent control log and support real-time control
NFR-02	Logging Frequency	JSBSim updates aircraft state 60 times per second so the telemetry pipeline must process data at the same frequency
NFR-03	Data Integrity	The system must flag and quarantine corrupted data to maintain data integrity so zero packet loss must be ensured to guarantee stable training.
NFR-04	Scalability	Data pipeline must support large number of telemetry parameters to ensure that the system can handle more complex flight models without redesigning the architecture.
NFR-05	Transparency	System must log data components separately

CHAPTER 4: System Design

This section provides system architecture, use-case diagram, sequence diagram and activity diagram to ensure clarity and efficiency in the system implementation.

4.1 System Architecture Diagram

a. Data Flow Architecture

This diagram shows how telemetry data moves through the system from source to consumption.

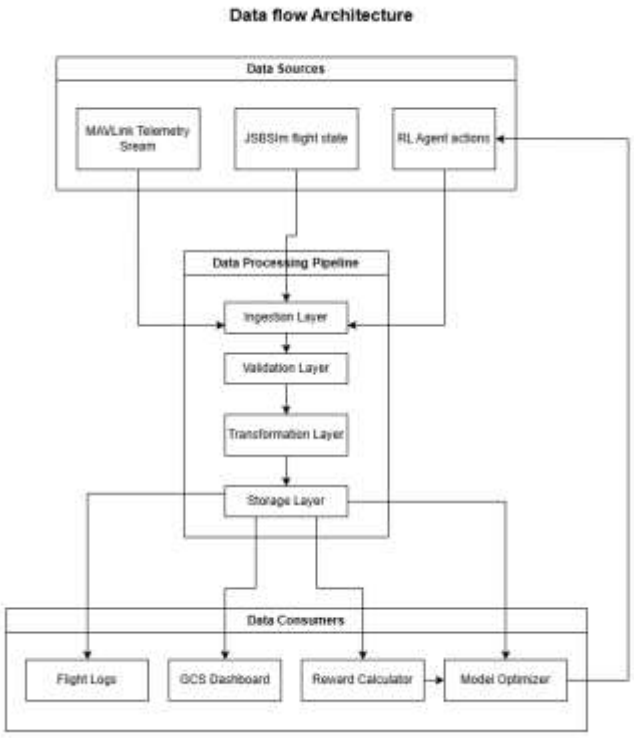


Figure 1: Data Flow Architecture

Data Sources Layer:

- MAVLink Telemetry Stream: Provides real-time flight data generated by JSBSim at 60Hz frequency.
- JSBSim Flight State: It provides six-degree-of-freedom physics data like: aircraft position, aerodynamic forces that forms the physical state of aircraft.
- RL agent actions: It generates control decisions like throttle adjustments, roll or yaw inputs for reward calculation and model optimization.

Data Processing Pipeline:

- Ingestion Layer: This layer receives raw MAVLink packets from the data sources and route them to internal processing pipeline. It's basically a entry point of the data system.
- Validation Layer: In this layer the entered data are detected to see if there is issues such as missing telemetry values, corrupted packets and null fields so that data integrity is ensured.
- Transformation Layer: After validation, the data is converted into suitable format like JSON for dashboard, CSV for offline analysis and binary format for high-speed telemetry processing.
- Storage Layer: Data is written into time-series database

Data Consumers: After processing the data is used by several system components like flight logs stores offline analysis and used for debugging, GCS Dashboard displays real-time flight

data, reward values are used for training AI agent and logged data is used to tune reward weights and improve policy convergence.

Feedback loop: After analyzing performance, the model optimizer generates updated policy which is sent back to RL agent and this continuous feedback process enables iterative learning and improvement.

b. System Architecture

This diagrams shows five architectural layers and their interactions.

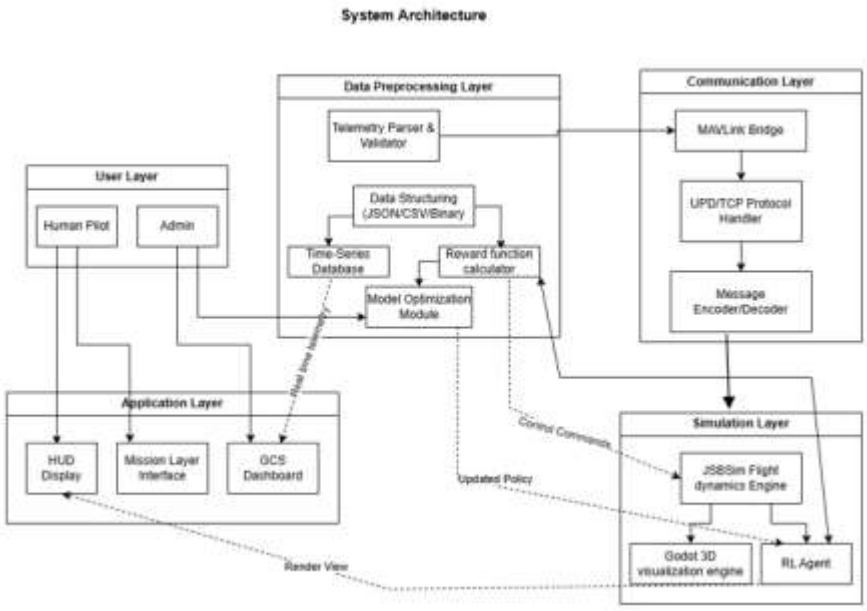


Figure 2: System Architecture

User Layer:

- Human Pilot: Can monitor aircraft status, manually control when needed and interacts with the system through HUD interface
- Admin: Manages system works like data access permissions, telemetry logging configuration.

Application Layer:

- HUD Display: Shows critical flight information like altitude, speed that helps pilot maintain situational awareness.
- Mission Layer Interface: Manages navigation task like waypoint planning, mission execution
- GCS Dashboard: provides analytical insights of flight performance like telemetry trends, AI reward metrics

Data Preprocessing Layer:

- Telemetry Parser & Validator: Extracts MAVLink messages and verifies their integrity.
- Data Structuring: Raw telemetry data is converted into structured format like binary logs, CSV/JSON.
- Time-Series Database: Stores high-frequency flight data
- Reward Function Calculator: Calculates reward function used in RL component wise
- Model Optimization Module: Tunes the reward weight to improve policy convergence and flight performance.

Communication Layer:

- MAVLink Bridge: Translate telemetry between JSBSim and rest of the system using MAVLink Protocol
- UDP/TCP Protocol Handler: UDP is used for high-speed telemetry and TCP for reliable command transmission
- Message Encoder/ Decoder: Encodes outgoing MAVLink packets and decodes incoming messages.

Simulation Layer:

- JSBSim Flight Dynamics Engine: Simulates aircraft physics with high fidelity at 100Hz frequency
- Godot 3D visualization engine: Renders 3D flight environment and interface elements at 60Hz
- RL agent: Generates autonomous control decisions

4.2 Use Case Diagrams

a. Use Case Diagram for Human Pilot

Figure 3: Use Case: Human Pilot

This use case diagram demonstrates the operational interactions between human operator and GCS.

- **Core Interaction:** Pilot can monitor flight data and interact with the telemetry dashboard using HUD.
- **Handover Logic:** Pilot can switch between manual control and AI-assisted autonomous control which ensures human oversight and safety

b. Use case diagram of AI co-pilot

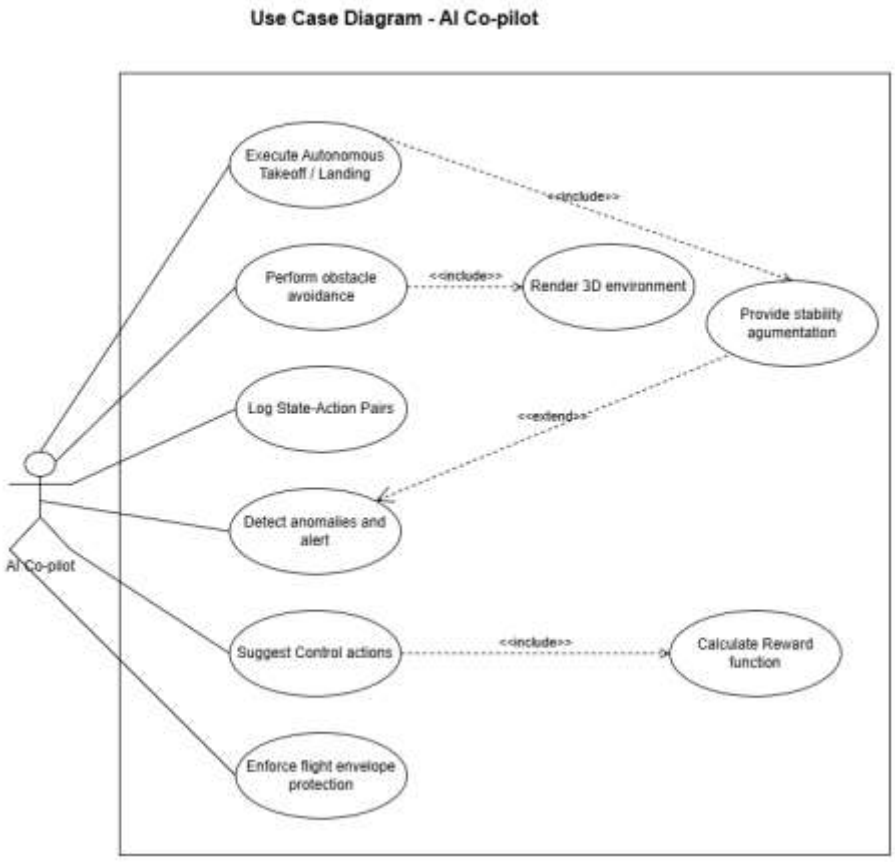


Figure 4: Use Case Diagram: AI co-pilot

This use case demonstrates behavior of RL agent as AI co-pilot showing how the AI agent interacts with system.

- **Core Interaction:** AI-co-pilot can perform autonomous flight task like autonomous takeoff and landing, flight stabilization and obstacle avoidance.
- **Safety and protection:** AI co-pilot continuously monitors the flight envelope to prevent unsafe flight behavior by generating penalties through reward function.

c. Use case diagram of System Admin

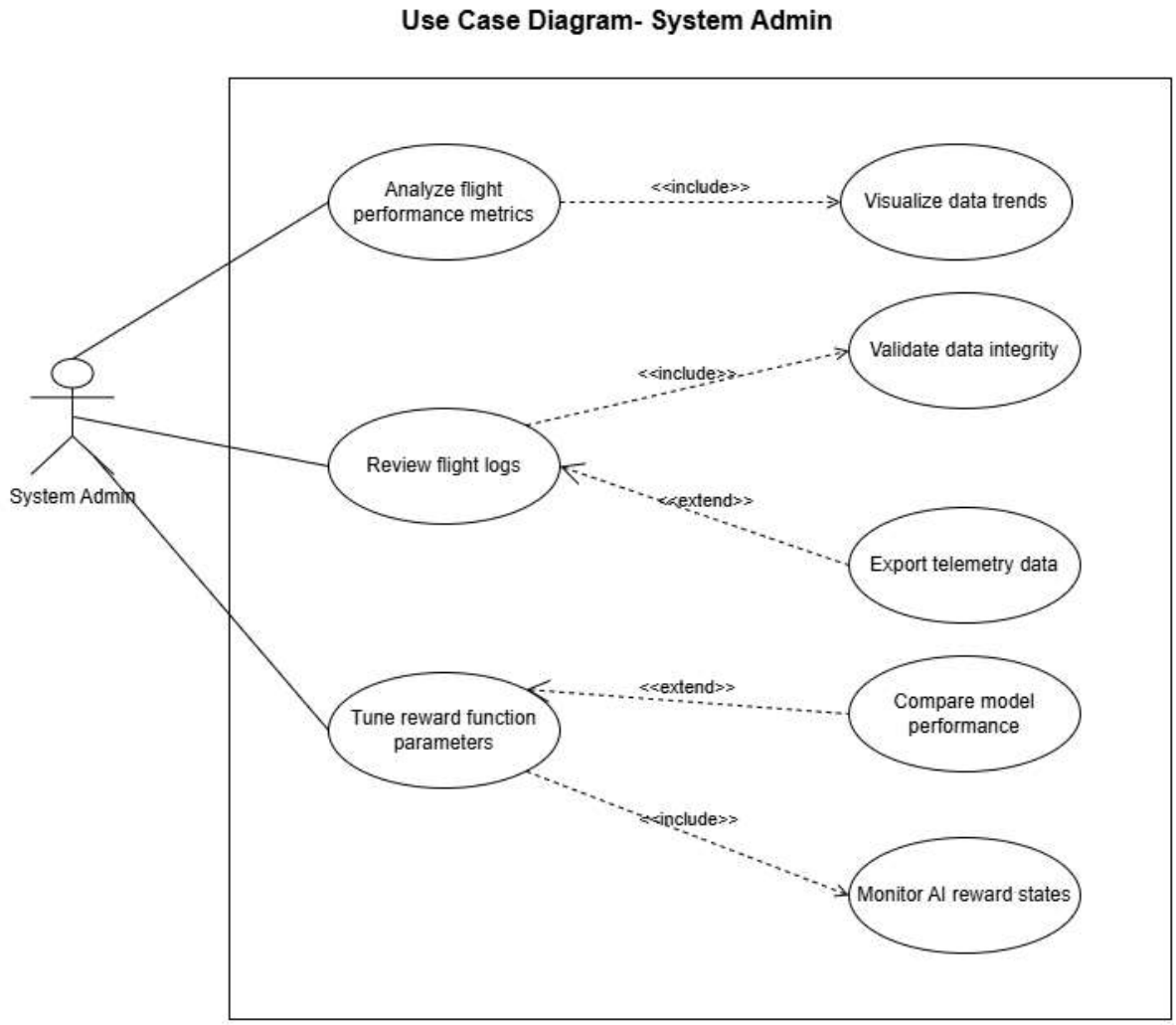


Figure 5: Use Case Diagram: System Admin

This use case focuses on actor System Admin who oversees the technical operations of the system.

- **Core Interactions:** They can perform many system functions like reviewing flight logs, analyzing performance metrics.
- **Data life-cycle:** They can manage entire telemetry data lifecycle by ensuring data integrity and exporting data for further research.

d. Use case diagram of System

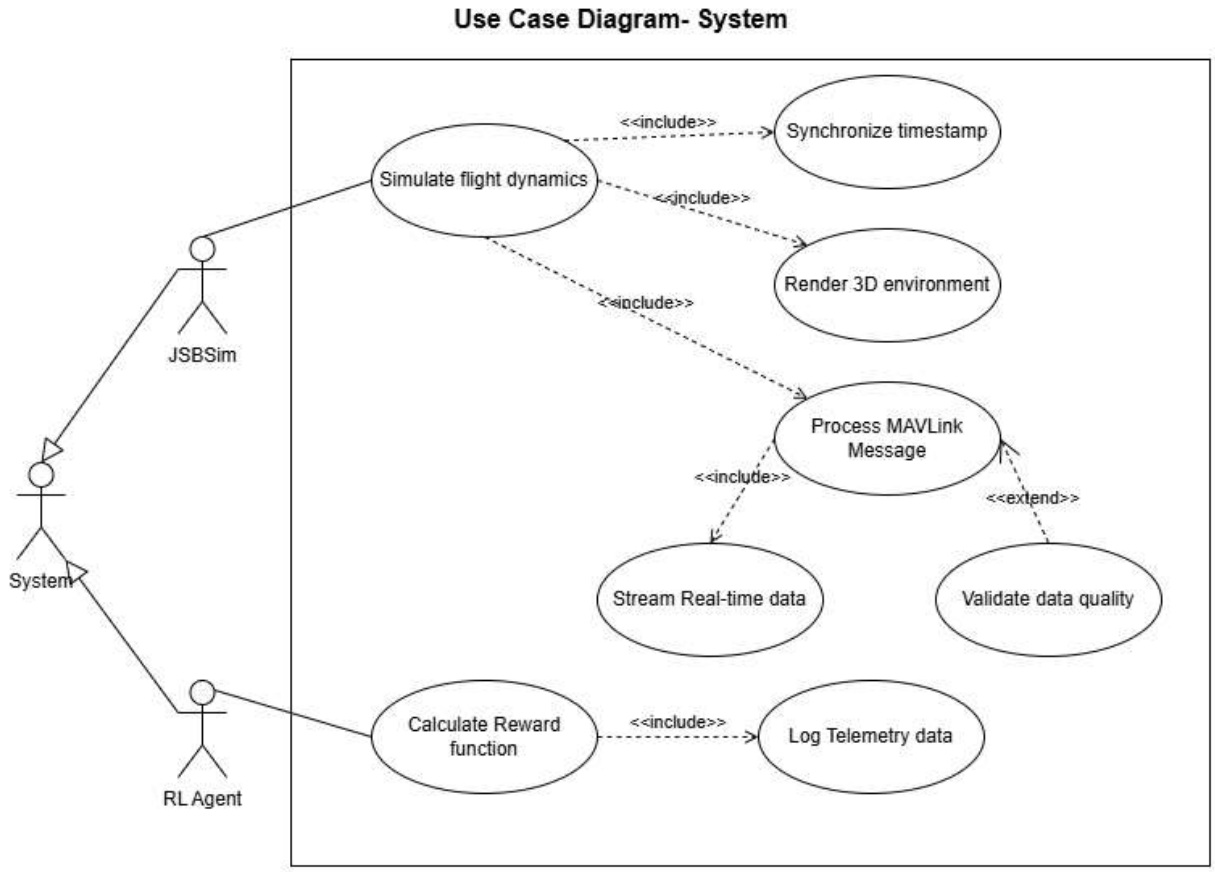


Figure 6: Use Case Diagram: System

This use case diagram demonstrate the interaction between core software system automatically without direct human input.

- **Core Interactions:** JSBSim simulates flight data at high frequency to produce realistic flight behavior and renders 3D environment via Godot. The generated telemetry is transmitted through MAVLink communication protocol. The system must also maintain timestamp synchronization to ensure telemetry data, visual rendering and AI decisions are aligned in time.

4.3 Activity Diagrams

a. Manual Flight control

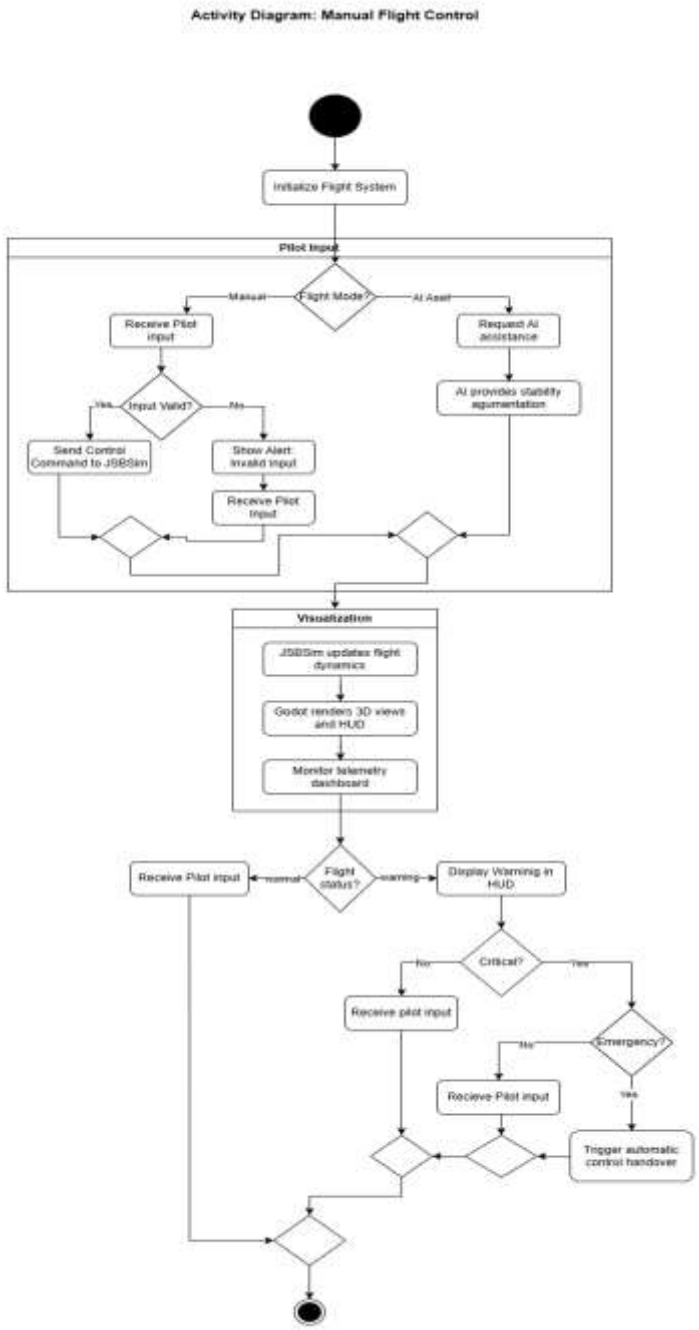


Figure 7: Activity Diagram- Manual Flight Control

- **System Initialization:** Firstly, system is initialized and then it waits for pilot input.
- **Pilot Input Processing:** System can choose flight mode (Manual or AI assisted). In manual mode, system receives pilot input, validates it and sends command to JSBSim. In AI

Assisted mode, pilot request AI assistance to maintain stability while maintaining pilot authority.

- **Validation & Monitoring:** JSBSim updates flight data, Godot renders 3D and HUD display and telemetry dashboard displays real-time flight status.
- **Flight status monitoring:** System constantly checks the flight status whether its normal or in a warning state and if in warning state it checks if the condition is critical or non-critical. In case of critical situation, if emergency is detected automatic control handover to AI, if not continues with pilot input.

b. AI co-pilot Stability Agumentation

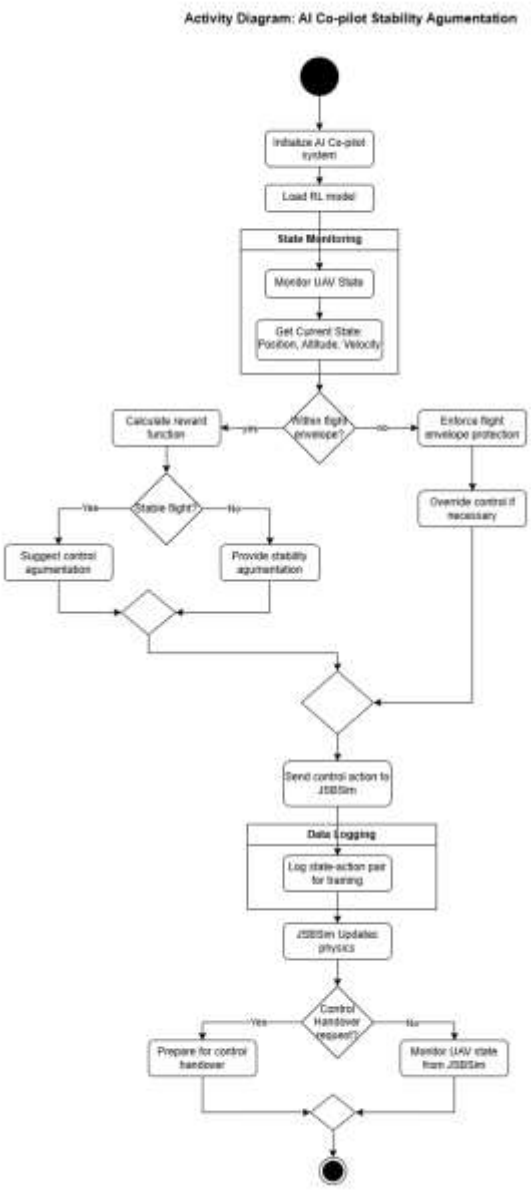


Figure 8: Activity Diagram: AI co-pilot Stability Agumentation

- **Initialization:** The system initializes AI co-pilot and loads RL model.
- **State monitoring:** System monitors UAV state and checks if UAV in within flight envelope.
- **Decision logic:** If outside envelope then system overrides control when necessary. If within envelope then in case of stable flight suggests control augmentation and in case of unstable flight provides active stability augmentation.
- **Execution & logging:** Sends control action to JSBSim and logs state-action paris for RL training.
- **Handover Check:** If requested then prepares for handover and if not continues monitoring UAV state.

c. Control Handover

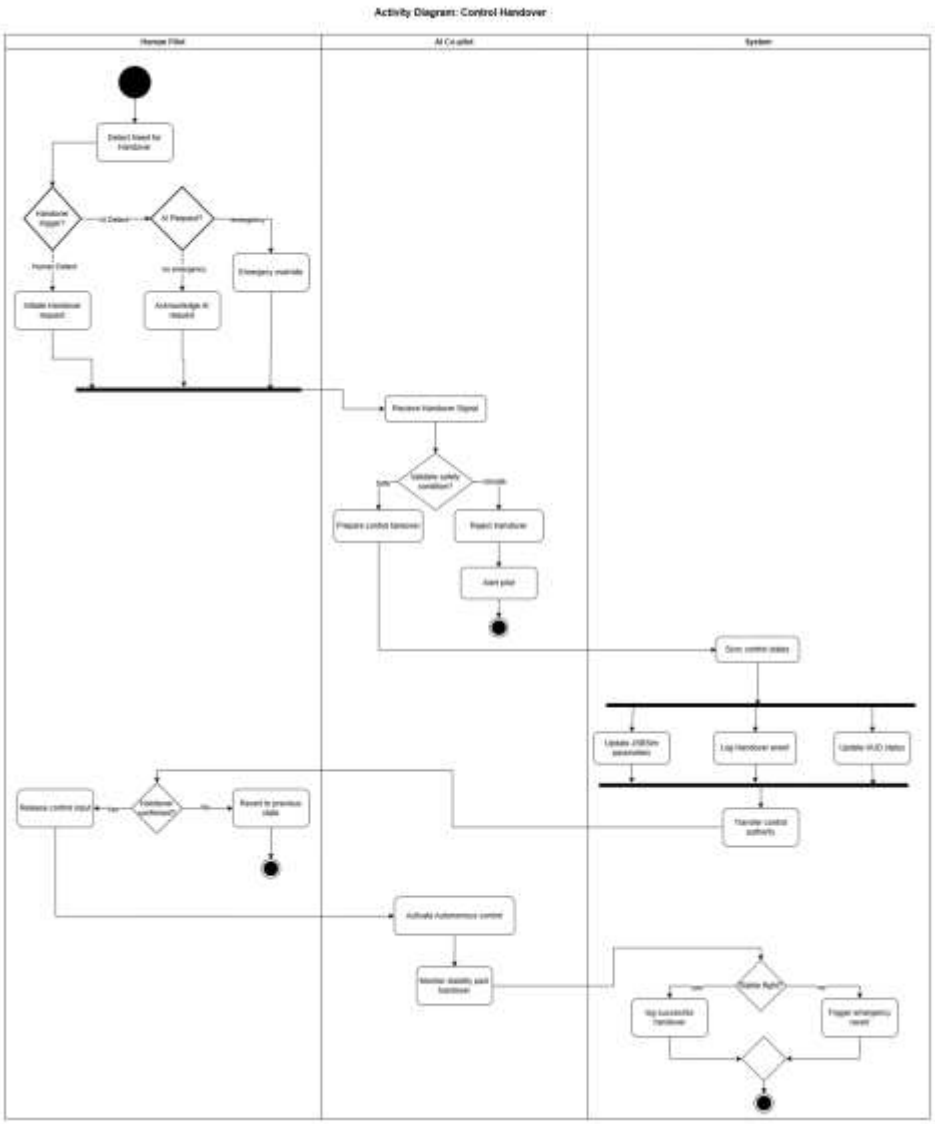


Figure 9: Activity Diagram: Control Handover

- **Handover initiation:** Human pilot can request for AI assistance when needed or when in emergency, there is AI co-pilot will override.
- **Safety Validation**: Safety conditions are checked before accepting control and rejects handover and alerts pilot if the conditions are unsafe.
- **System synchronization:** Updates the JSBSim parameters and logs handover events for critical analysis and also updates HUD to show the authority change.
- **Control transfer:** After confirmation, the control authority is transferred to AI co-pilot.
- **Post-Handover Monitoring:** After handover, the system checks flight stability.
- **Outcome logging:** If flight remains stable, successful handover is logged and in case of instability, triggers emergency revert to previous state.

d. Reward function design and Model optimization

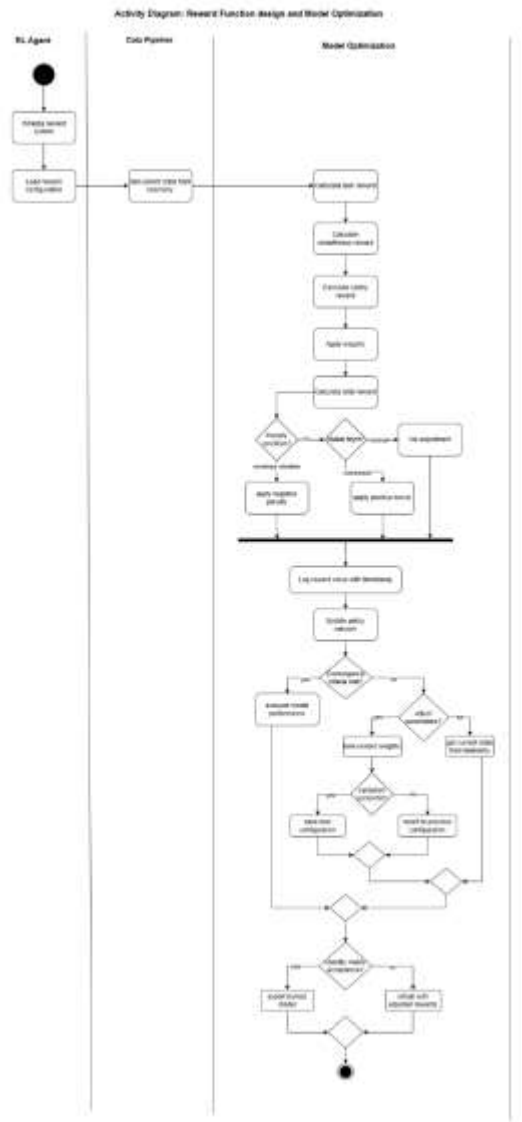


Figure 10: Activity Diagram- Reward function design and Model Optimization

- **Initialization:** RL agent loads reward configuration and data pipeline retrieves current flight state from telemetry.
- **Reward calculation:** Calculates task, smoothness, and safety rewards and applies weights to compute total reward
- **Decision Logic:** In case of penality condition, negative penalty is given and if there is stable flight then positive bonus is applied.
- **Logging & Updates:** Reward values are logged along with timestamp for transparency
- **Optimization Loop**: Convergence criteria is checked and reward weights is tuned then new configuration is validated. As per the validation success, configuration is saved or reverted.
- **Final Output:** Can also export trained model if stability metrics are acceptable

e. Telemetry Data pipeline and logging

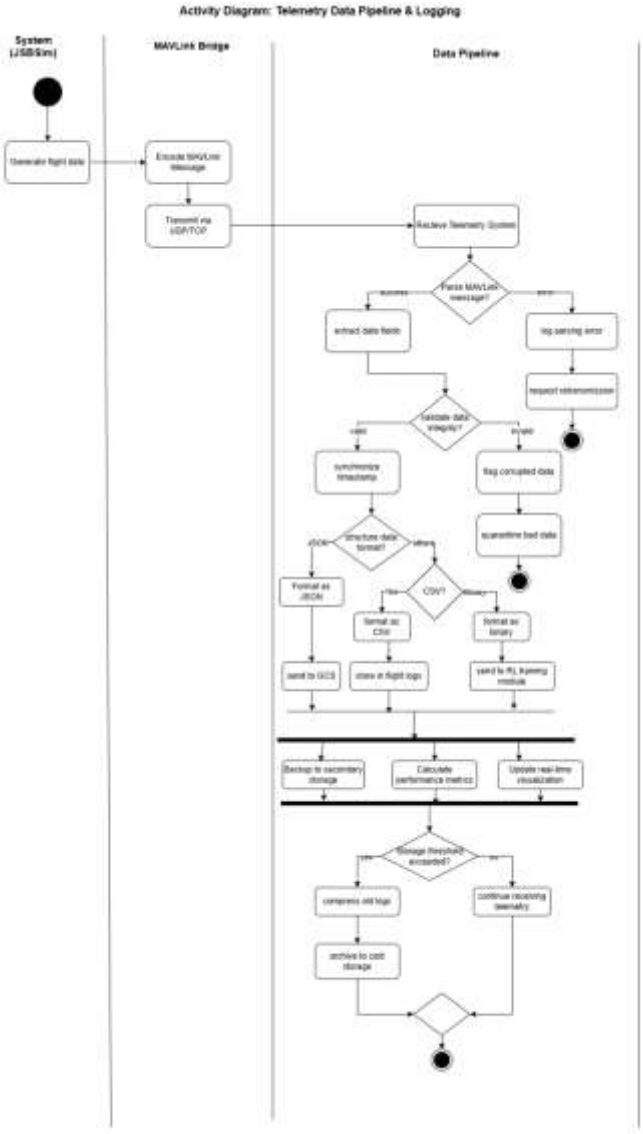


Figure 11: Activity Diagram- Telemetry Data Pipeline and Logging

- **Data transmission:** JSBSim generates flight data and after it is encoded, data is transmitted via UDP/TCP protocols.
- **Data validation:** Errors are logged and retransmission is requested. If parsing is successful data fields are extracted and data integrity is validated. If invalid data, corrupted data is flagged and quarantines bad packets. If valid data, timestamp synchronization is done.
- **Data structuring:** Stored in JSON to sent to GCS dashboard, stored in CSV for offline analysis and stored in binary for RL training module
- **Parallel processing:** System performs 3 operation concurrently: backing to secondary storage, calculating performance metrics and updating real-time visualization.
- **Storage Management:** Checks if storage threshold is exceeded and if exceeded old logs are compresses and archives to cold storage.

f. Dashboard Visualization

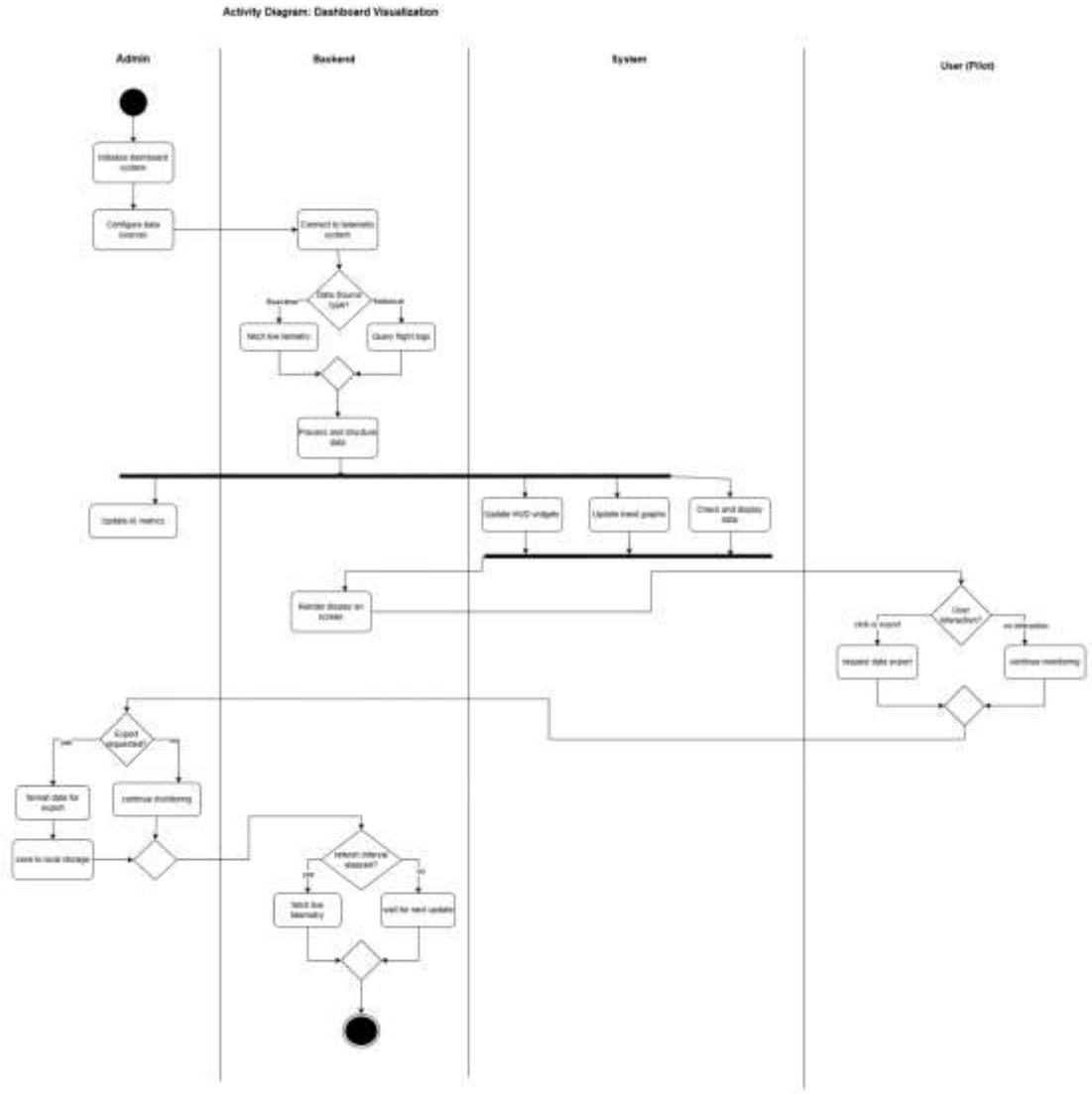


Figure 12: Activity Diagram- Dashboard Visualization

- **Initialization and Data Retrieval:** Admin initializes dashboard, backend connects telemetry system and fetches live telemetry data and processes and structures data for visualization.
- **Parallel Visualization:** System updates HUD widgets, trend graphs and checks and display data alerts and renders the final display on screen for Pilot
- **User Interaction:** If export is requested, backend formats data and saves to local storage and if not, real time monitoring is continued.
- **Refresh loop:** System checks if refresh interval has elapsed and fetches new live telemetry or wait for next update.

4.4 Sequence Diagrams

a. Telemetry Data Pipeline and logging

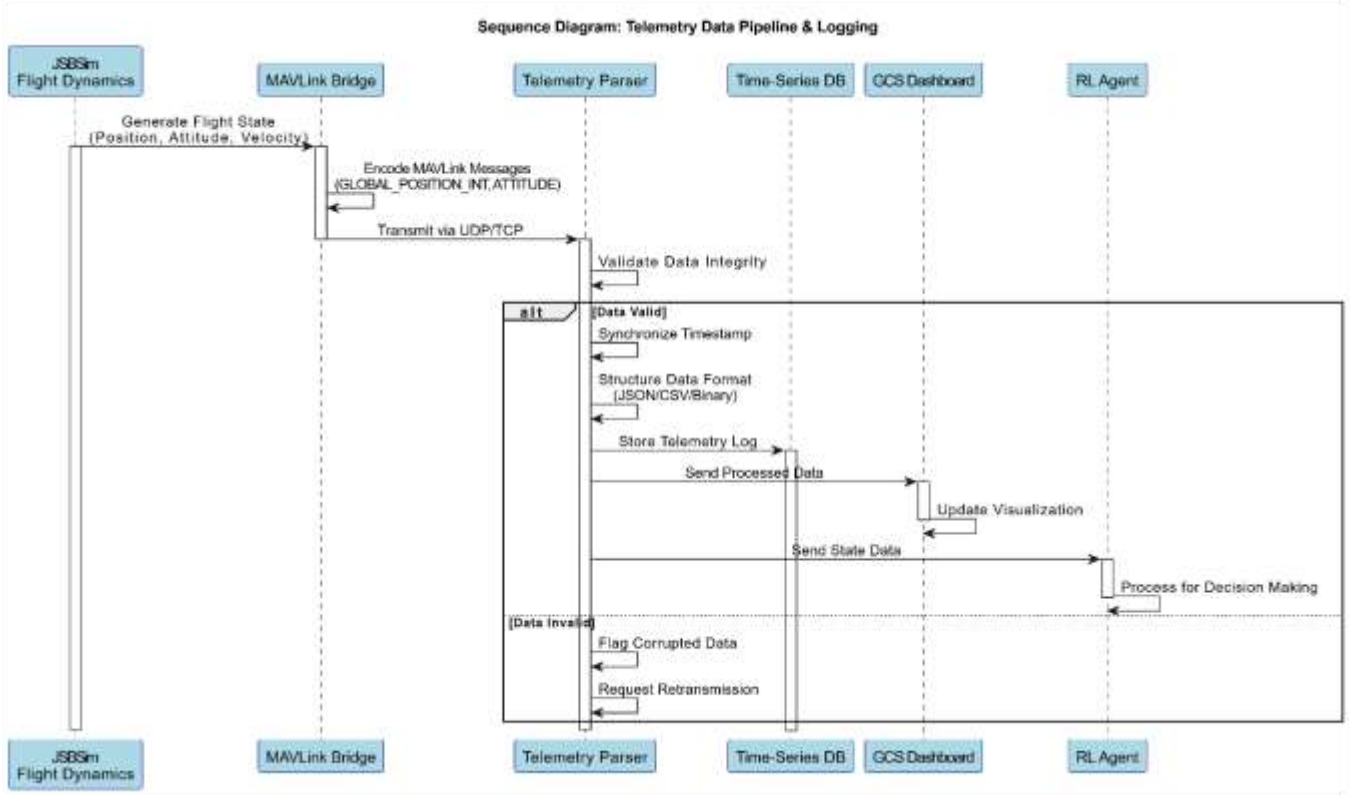


Figure 13: Sequence Diagram- Telemetry Data Pipeline and logging

- **Data generation:** JSBSim generates flight data and MAVLink encodes message and that is transmitted by UDP/TCP.
- **Data processing:** Telemetry parser recieves and validates the data by synchronizing timestamp, structuring them into JSON/CSV/Binary format and flagging corrupted data and requesting retransmission.
- **Data Distribution:** Processed data is then sent to GCS dashboard for visualization and state data is sent to RL agent, while other telemetry logs are stored in Time-Series DB for historical analysis.

b. Reward Function Calculation & Model Optimization

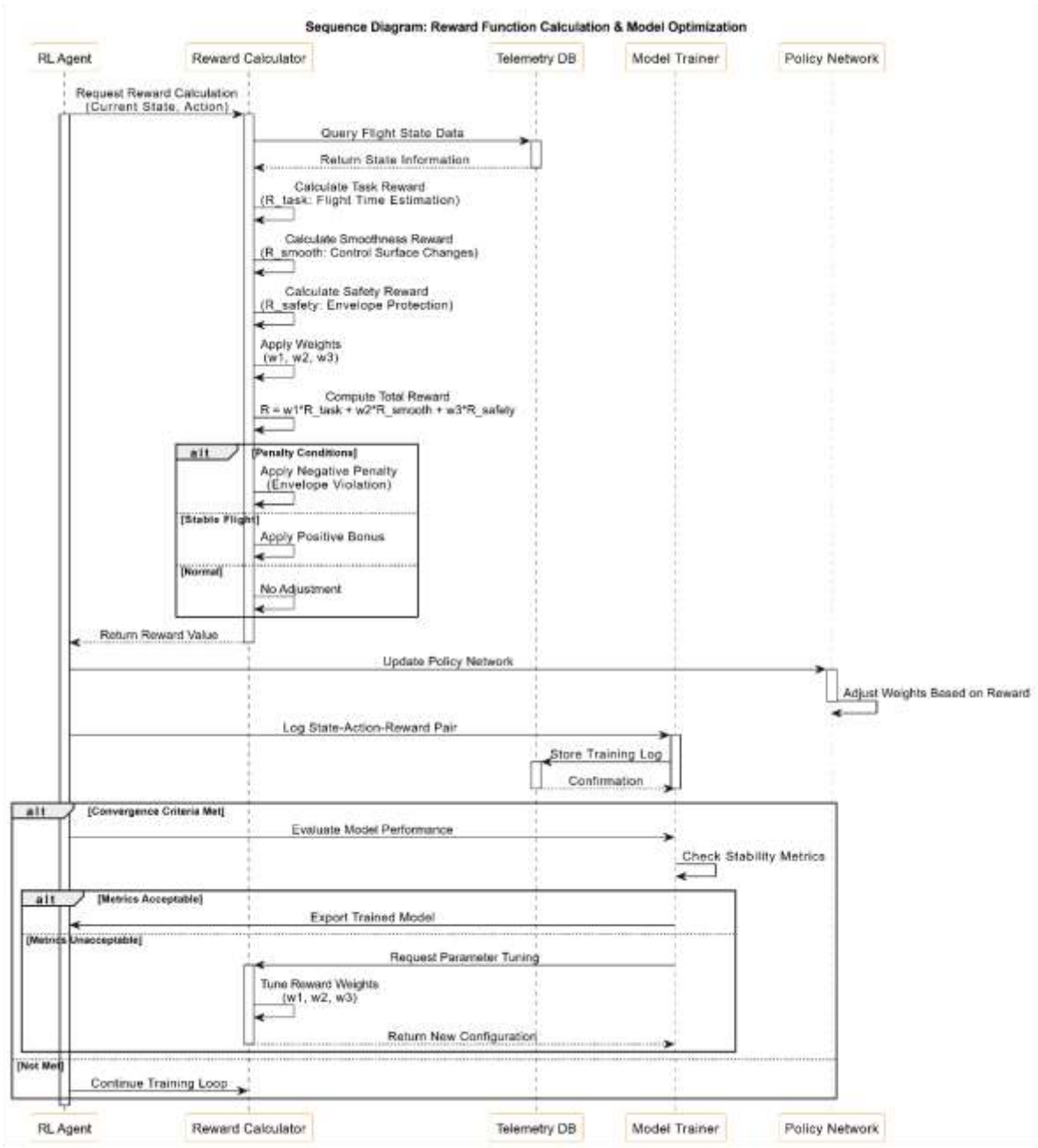


Figure 14: Sequence Diagram- Reward Function calculation and Model Optimization

- **Reward Calculations:** After getting the flight state data, task, smoothness and safety rewards are computed separated and then weights (w1, w2, w3) are applied to calculate total reward.
- **Penalty logic:** If there is instability or envelope violation, then negative penalty is applied and positive bonus is added if stable flight.

- **Model Optimization:** State-action reward pairs are logged for training and if convergence is met, model performance is evaluated and if not met, reward weights are tuned and training loop continues.

c. Control Handover between Human Pilot and AI co-pilot

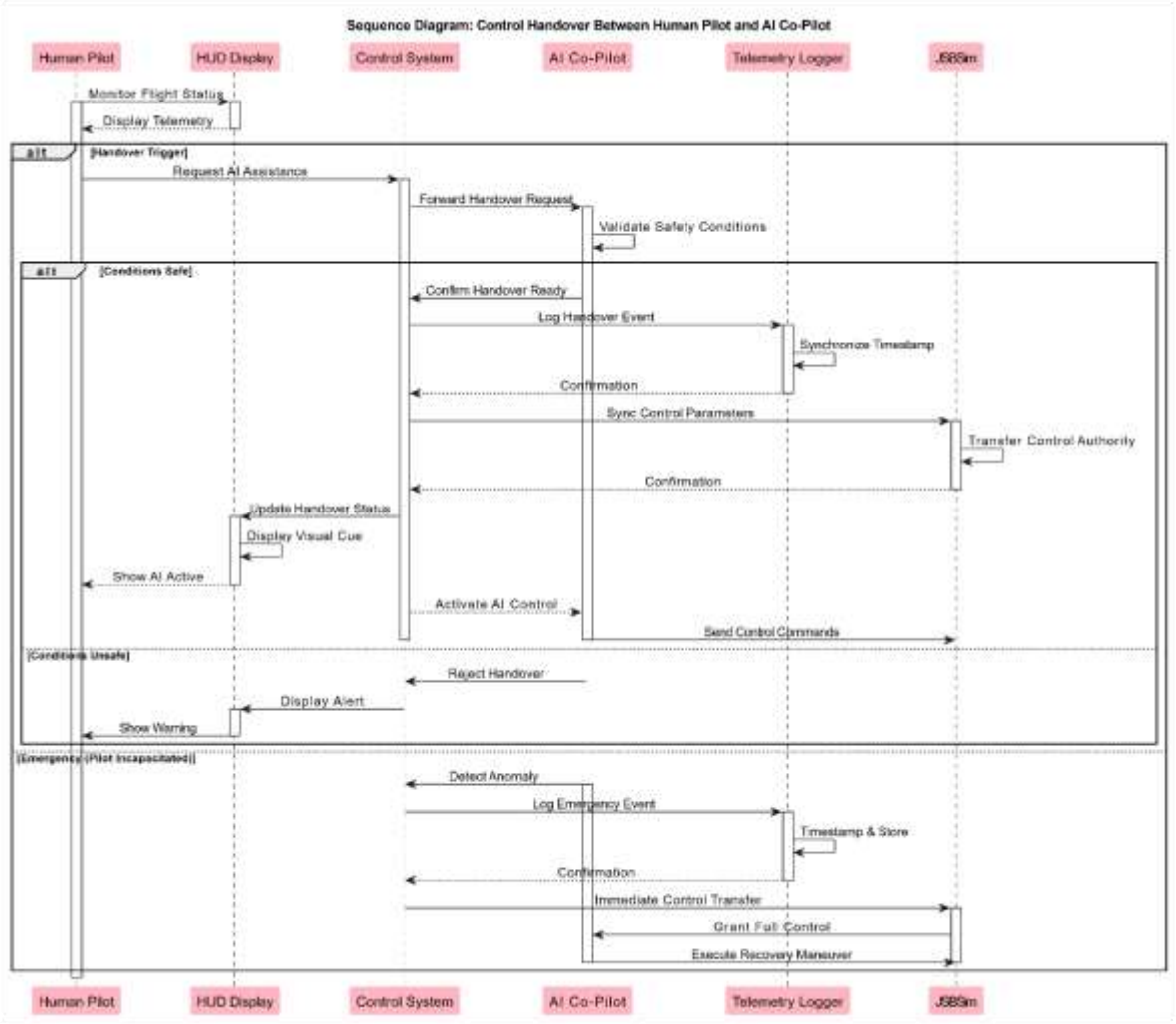


Figure 15: Sequence Diagram: Control Handover between Human pilot and AI co-pilot

- **Handover Initiation:** Pilot can request AI assisted mode when needed.
- **Safety Validation:** Before accepting the control, AI co-pilot validates the safety. If the condition is safe, it confirms handover and log the events but in case of unsafe condition handover is rejected and alert is displayed to pilot.
- **Control Transfer:** AI activates autonomous control and sends commands to JSBSim. Also HUD updates and visual cues are displayed to pilot.
- **Emergency handling:** If emergency is detects, it logs the event with timestamp and immediate control is transferred to AI for recovery maneuver.

d. System Initialization and Startup Sequence

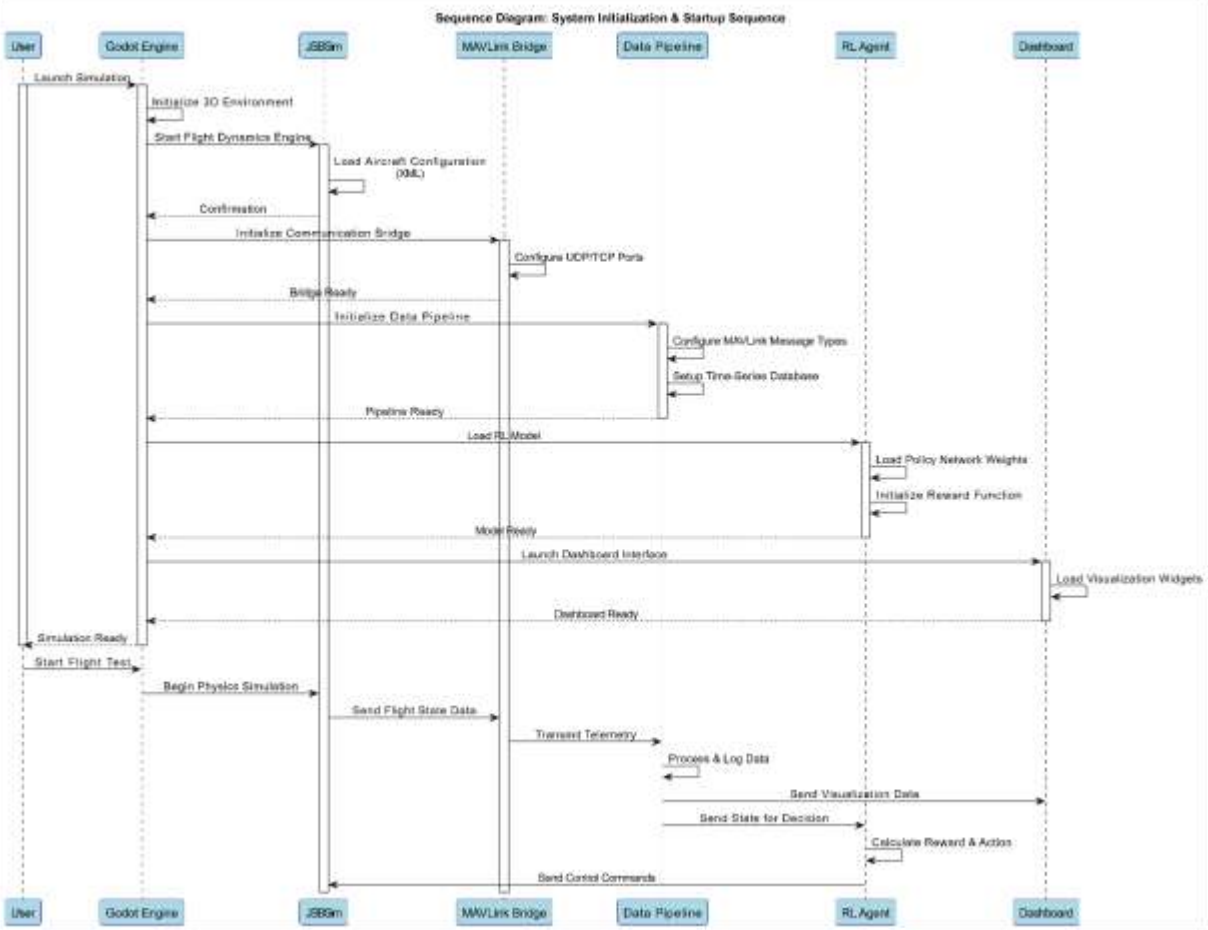


Figure 16: Sequence Diagram- System Initialization and startup sequence

- **System launch:** 3D environment is initialized in Godot engine and then JSBSim starts flight dynamics and loads aircraft configuration.
- **Communication setup:** MAVLink initialized UDP/TCP port configuration.
- **Model loading:** Policy network weights are loaded by RL agent and reward function is initialized. Visualization widgets is loaded in GCS dashboard.
- **Flight test:** After all the components are ready, user starts flight test and JSBSim begins the simulation by sending flight data, data pipeline processes it can logs those telemetry then RL agent calculates reward and sends to control commands to JSBSim.

References

Aziz, F., & Loya, A. (2025). Multi-Mode MAVLink Test Bench AeroQT for Aircraft Simulation and Real-Flight Validation. *Aerospace Research Communications, 3*.

Defense Technical Information Center. (2022). *Telemetry Data Mining for Unmanned Aircraft Systems.* Defense Technical Information Center.

Hansberger, J. T., Meacham, S., & Blakely, V. (2018). Designing Data & Dashboard Visualizations for Future UAV Systems. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting. 62*, pp. 177-181. Los Angeles: SAGE Publication. Retrieved from https://journals.sagepub.com/doi/abs/10.1177/1541931218621044

Hubsky, O. M. (2024). Analysis of User Interfaces for Ground Control Stations of Unmanned Aerial Vehicles. *Cybernetics and Computer Engineering*, 5-23. doi:https://doi.org/10.15407/kvt217.03.005

Jovanovic, M., & Starčević, D. (2008). Software Architecture for Ground Control Station for Unmanned Aerial Vehicle. *2008 UKSim 10th International Conference on Computer Modeling and Simulation* (pp. 284-288). IEEE Xplore. doi:10.1109/UKSIM.2008.12

Khanzada, H. R., Maqsood, A., & Basit, A. (2025). *Reinforcement learning for UAV flight controls: Evaluating continuous space reinforcement learning algorithms for fixed-wing UAVs.* PLOS ONE. doi:https://doi.org/10.1371/journal.pone.0334219

Poma, A., Sojo, A., Maza, I., & Ollero, A. (2025). Ground Control Station for Multi-UAV Systems in Infrastructure Inspection and Environmental Monitoring Applications. *Journal of Intelligent & Robotic Systems,, 111*, 109. doi:https://doi.org/10.1007/s10846-025-02234-1

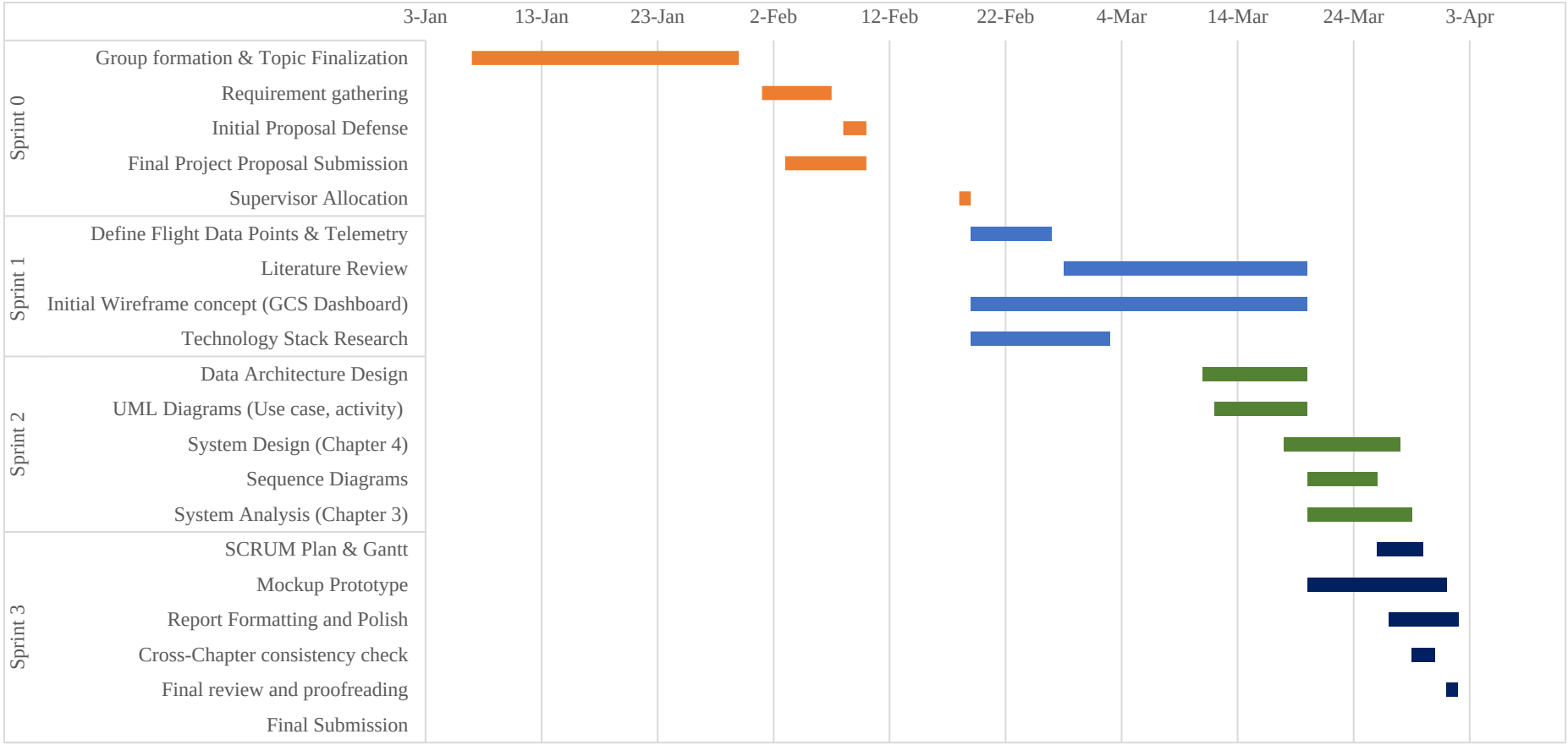
Richter, D. J., Calix, R. A., & Kim, K. (2024). *A review of reinforcement learning for fixed-wing aircraft control tasks.* IEEE Access. doi:https://doi.org/10.1109/ACCESS.2024.3428765

Sivamani, G. K., & Gudipalli, A. (2024). Design and implementation of DATA logging and stabilization system for a UAV. *10*(4). doi:https://doi.org/10.1016/j.heliyon.2024.e26394.

Tovarnov, M. S., & Bykov, N. V. (2022). Reinforcement learning reward function in unmanned aerial vehicle control tasks. *Journal of Physics, 2308*(1).

Ufelek, M., Yoruk, O., & Cakici, M. E. (2020). Real Time Data Visualization in Telemetry Systems. *European Test and Telemetry Conference (ETTC 2020)*, (pp. 107-112).

Appendix



Meeting Minutes